

claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-a74b26c168959972d.jsonl`  
- SHA-256: `3f58477b491209bead944b13db086e381fc769ead6c4eb6dc32bd5fa0b1e3e82`  
- Source modified: `2026-06-19T15:55:38+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_ac0cbb29-6bf`  
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`
```

Transcript

1. user (2026-06-19T15:52:06.169Z)

You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements that CONTRADICT the current code or data.

HARD RULES:

- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured, say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.

Working dir is the repo root; use Read/Grep/Glob freely.

ADVERSARIALLY VERIFY these proposed staleness fixes for ONE file: docs/QUALITY_ITERATIONS.md. For EACH proposed fix: (1) open docs/QUALITY_ITERATIONS.md and confirm stale_text appears VERBATIM (tighten/expand it to an exact, UNIQUE match if needed ? this text will be applied by exact string match, so it must be perfect); (2) re-open the cited code yourself and confirm the claim is genuinely contradicted by the CURRENT code; (3) confirm corrected_text is accurate, preserves voice, and is minimal; (4) DROP it if it is subjective, an owner-gated decision, already-correct, or you cannot verify the evidence. Output ONLY the survivors (verdict confirm or revise), with stale_text guaranteed to be an exact unique substring of the file.

PROPOSED FIXES:

```
[  
  {  
    "file": "docs/QUALITY_ITERATIONS.md",  
    "stale_text": "***Golden corpus = the 6 labelled matches** in  
`output/human_lovo_manifest.json` (owner-confirmed\n2026-06-13: use all 6 ? `adarsh_avi_singles,  
kushagra_singles, largetest_doubles, gbaaddy,\ntestlarge_short, mahadevpura_singles`)."",  
    "corrected_text": "***Golden corpus = the 15 labelled matches** in  
`output/human_lovo_manifest.json` (the original 6 ?\n`adarsh_avi_singles, kushagra_singles,  
largetest_doubles, gbaaddy, testlarge_short,\nmahadevpura_singles` ? owner-confirmed 2026-06-13,  
since grown to 15 with the multicourt/Boxhill clips).",  
    "why_stale": "States the present golden corpus is 'the 6 labelled matches' and enumerates  
only the original 6, but output/human_lovo_manifest.json now has 15 entries. This is a present-  
tense definitional claim about the current corpus (the doc's own later entries already cite  
'n=15 golden' at line 62 and 'n=13' at line 109).",  
    "severity": "high"  
  },  
  {  
    "file": "docs/QUALITY_ITERATIONS.md",
```

```

    "stale_text": "> retired from the box (their labels are gone); the present golden set is **9
multicourt clips**. On that",
    "corrected_text": "> retired from the box (their labels are gone); at the time of this re-
grounding the golden set was **9 multicourt clips** (since grown to 15). On that",
    "why_stale": "Present-tense claim 'the present golden set is 9 multicourt clips' is now
wrong on both counts: output/human_lovo_manifest.json has 15 entries, and the set is not all-
multicourt (the P7 stratified table at line 66-68 reports 11 single-court + 4 multicourt). The
measured  $F1 \approx 0.034$  in this note was genuinely computed on the n=9 set, so only the present-
tense size assertion is corrected (re-cast as the corpus state at re-grounding time) to preserve
the measurement's provenance.",
    "severity": "medium"
}
]

```

2. assistant (2026-06-19T15:52:08.484Z)

I'll verify each proposed fix by checking the actual file content and the cited evidence.

3. user (2026-06-19T15:52:11.982Z)

Already up to date.

4. user (2026-06-19T15:52:12.351Z)

```

<class 'list'>
15

```

5. assistant (2026-06-19T15:52:14.739Z)

The manifest has 15 entries. Now let me confirm the verbatim text in the doc and re-read the relevant code/data.

6. user (2026-06-19T15:52:15.904Z)

```

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-600 of 749 total (26506 tokens, cap
25000). Call Read with offset=601 limit=600 for the next page, or Grep to find a specific
section. Do NOT answer from this page alone if the answer may be further in the file.]</system-
reminder>

```

```

1      # Rally-detection quality iterations ? living log
2
3      **Purpose.** A running, reverse-chronological log of how we evolve rally-detection
quality: each
4      iteration's hypothesis, what shipped, the measured (or expected) effect, and the lesson.
This is the
5      **live** continuation of the historical trail in
6      [archives/quality-iterations/](archives/quality-iterations/README.md) ? per the
archive policy
7      (`CLAUDE.md`), **terminal-state** (DONE/REJECTED) work gets a dated subdir there; this
file stays at
8      `docs/` top level and keeps growing, then its entries graduate to the archive when they
conclude.
9
10     **Methodology (non-negotiable).** Every quality change rides the experiment harness
11     ([EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)): new cues land **flag-gated, default
OFF**; they're
12     graded **offline against golden labels** (per-video + LOVO-honest); promotion to default
happens **only
13     on measured lift** through the `run_regression.py` gate; **rollback = flip a config
flag, never
14     `git revert`.** Cues are designed in
[MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); scoring
15     in [EVALUATION.md](EVALUATION.md); the 2-layer model in
16     [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).

```

```

17
18     **Golden corpus = the 6 labelled matches** in `output/human_lovo_manifest.json` (owner-
confirmed
19     2026-06-13: use all 6 ? `adarsh_avi_singles, kushagra_singles, largetest_doubles,
gbaaddy,
20     testlarge_short, mahadevpura_singles`). The model trained on these is what we run
alongside the existing
21     detector on NEW footage for the side-by-side review.
22
23     ---
24
25     ## Baselines that drive strategy (IoU 0.5 vs human golden labels)
26
27     | Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
28     |---|---|---|
29     | Heuristic (velocity windowing) | 0.657 | 0.157 |
30     | Gen-0 owned head (LOVO, n=6) | 0.744 | 0.561 |
31     | Two-stage WASB+Gemini refine | 0.83 | ? |
32     | Gemini wrapper (`gemini-flash-latest`) | 0.93 | 0.66 |
33
34     **Reading:** clean footage is commoditized (serve via Gemini); **multi-court drops the
commodity wrapper
35     to 0.66 ? that is the owned model's ship target.** Baselines tracked in
`eval_baselines/`. The owner's
36     observed false positive ? a player **holding/flicking** the shuttle between points
counted as a rally ?
37     is the precision bug the fusion gates target.
38
39     ---
40
41     ## Current track (live) ? newest first
42
43     ### P7 ? Long-rally (>5s) quality lens (eval) + reel-selection knob ? MEASURED, default-
OFF *(2026-06-18)*
44     **Hypothesis (owner).** Local OVER-FIRES, and its false positives are mostly SHORT
(motion blips,
45     background-court fragments on multicourt). Filtering the eval to **long rallies (>5s)**
? the
46     eye-catching highlights that carry the product ? should strip most short FPs and reveal
local's real
47     long-rally quality, answering the North-star "do we beat Gemini on the single-match
long-rally happy
48     path?". **Duration** (end?start), NOT shot-count, is the filter: golden `shots_count` is
blank and the
49     no-AI path reports it "Unknown", whereas duration comes from the start/end labels we
already trust.
50
51     **What shipped (both default-OFF, zero behaviour change at the defaults).**
52     - **Eval lens** ? `rally_seg_eval.py --min-duration <s>` filters BOTH predictions AND
golden GTs to
53     rallies longer than `s` *before* matching, consistently across the tIoU and R2
tolerance paths (a
54     FILTER, not a new metric). Plus `--stratify` / `--long-tau` (default 5s): a `{all vs
long} x {single
55     vs multicourt}` aggregate table. Court type = the committed `MULTICOURT_CLIPS` set
(the manifest is
56     gitignored, so the set is the auditable source), overridable by a manifest
`court_type` field; kept
57     in sync with [`GOLDEN_VIDEOS.md`](GOLDEN_VIDEOS.md).
58     - **Product knob** ? `stitching.min_rally_duration` (s, 0 = OFF) drops short rally pairs
before the
59     reel compile in `tools/generate_highlights.py` ? a cleaner reel-selection filter than
60     `min_shot_count` (which leans on the "Unknown" shot_count).
61
62     **Measured (n=15 golden, milestone cap6+R4, LOCAL vs golden; reproduce: `python -m

```

```

backend.eval.rally_seg_eval --stratify`):**
63
64 | stratum | court | n | tolF1 | tolP | tolR | tIoU F1@0.5 | seg_ratio |
65 |---|---|---|---|---|---|---|---|
66 | all | single | 11 | 0.375 | 0.326 | 0.464 | 0.482 | 1.434 |
67 | all | multicourt | 4 | 0.294 | 0.238 | 0.393 | 0.430 | 1.601 |
68 | all | all | 15 | 0.353 | 0.302 | 0.445 | 0.468 | 1.478 |
69 | **long>5s** | single | 11 | 0.264 | 0.278 | 0.267 | 0.416 | **1.037** |
70 | **long>5s** | multicourt | 4 | 0.240 | 0.197 | 0.310 | 0.430 | **1.597** |
71 | **long>5s** | all | 15 | 0.257 | 0.256 | 0.279 | 0.420 | 1.186 |
72
73 Pooled counts: single **608 preds / 416 GTs** (only 36% of preds are >5s vs 50% of GTs ?
long pred:GT ?
74 **218:210 = 1.04**); multicourt **288 preds / 179 GTs** (55% of preds >5s ? long pred:GT
= **158:100 =
75 1.58**).
76
77 **Findings ? the lens turned a one-line hypothesis into a 3-part measured map (NUMBERS
INFORM):**
78 1. ? **Single court, count-level: thesis HOLDS.** Local's over-production is dominated
by SHORT windows
79 ? restricting to >5s collapses seg_ratio **1.43 ? 1.04** (preds ? GTs in count). The
short-blip FPS
80 are real and duration-filterable.
81 2. ?? **Multicourt: thesis FAILS.** Even among >5s rallies local over-fires **1.58x** ?
adjacent-court
82 games are genuine LONG rallies ? long *spurious* windows. **A duration filter does
NOT clean
83 multicourt; that needs court-localization** (Gate-B / court mask ? gap-closure Lever
C). So "FPS are
84 mostly short" is a *single-court* statement.
85 3. ?? **Strict tolerance precision does NOT jump** (tolP single 0.326 ? 0.278) even
where the count
86 normalizes, because the >5s pred set is contaminated by **over-MERGED windows**
(kushagra med 9.9s,
87 mahadevpura_1's 63s blob, GX010141 14.8s) and cap=6 truncates genuinely-long rallies
so the strict
88 |?end| ? 1.5 s match fails (tIoU, forgiving, holds ~0.42; strict tol drops). The
long-rally
89 *strict-eval* happy path is gated by merge-control + the cap-vs-long tension, not
just FP-stripping.
90 4. ? **Product knob is still a clean reel win.** A reel doesn't need strict boundary
tolerance ? an
91 over-merged 15 s clip is a fine highlight, and dropping <5 s pairs removes the
single-court short
92 junk. Defaults OFF; owner reviews the numbers before any flip.
93
94 **Gemini:** not computable in-checkout ? the Boxhill gemini intervals live in the owner-
side scorer's DB
95 (not committed). The lens is metric-agnostic: feed Gemini's intervals through the same
96 `filter_by_min_duration` to get its long-rally row (owner-side one-liner). From gap-
closure §3, Gemini
97 already fires fewer/longer windows with ~0 short FPS, so a long filter mostly affects
LOCAL.
98
99 **Lesson.** A plausible product one-liner ("filter to long ? precision jumps") split
into three measured
100 truths once instrumented: single-court count ?, multicourt ? (needs a court signal),
strict-precision ?
101 (needs merge control). Default-OFF lens + knob; the owner decides any flip on the
numbers. *(P7 in
102 [ `RALLY_DETECTION_GAP_CLOSURE.md` ](RALLY_DETECTION_GAP_CLOSURE.md) §7.)*
103
104 ### R1 ? MILESTONE LAUNCHED (cap=6 + R4 flipped default-ON) ? re-verified at n=13
*(2026-06-18)*

```

105 The R1 milestone is ****shipped**** (PR #204): the W1 cap=6 + R4 inactivity-end defaults are now ON.

106 Full quality-impact table on both rulers, the over-seg guard, and the honest caveats are recorded in

107 the ****[R1 Milestone Launch Report](R1_MILESTONE_LAUNCH_REPORT.md)**** (per the Launch-Report convention).

108

109 ****Re-verified on the grown n=13 corpus**** (the 2 new Boxhill clips GX010137/GX010141) ? the milestone

110 holds and the headline is ***stronger*** than at n=11:

111

112 | transition | n=11 (filed) | ****n=13 (launched)**** |

113 |---|---|---|

114 | baseline ? milestone tolF1 | ?+0.232, 10/0, p=0.002 | ****?+0.222, 12/0/1, p=0.0005**** |

115 | cap12 ? cap6 tolF1 | ?+0.108, 10/0, p=0.002 | ****?+0.110, 12/0/1, p=0.0005**** |

116 | cap6 ? +R4 tolF1 | ?+0.047, 8/0, p=0.008 | ?+0.040, ****9/1/3****, p=0.0215 |

117 | net over-seg guard | PASS (mr 0.694?0.150) | ****PASS**** (mr 0.651?0.161, 0 regressions)

118

119 Both new clips improve (GX010137 0.448?0.685, GX010141 0.333?0.431); tIoU ladder baseline 0.236 ?

120 milestone 0.474. Independently reproduced by a 2nd harness (#214 `lovo\_resweep.py`, matches to 3 dp;

121 LOVO picks cap=6 ****13/13**** on tIoU) and survived a 5-lens adversarial workflow (every L00 fold clears

122 the bar; worst-fold drop GX010128 still ?+0.202, p=0.0010 ? no single clip drives it). ****Honest notes:****

123 W1 cap is the dominant lever / R4 is the marginal +0.040 increment (1 loss = GX010137, a tolerance

124 artifact); the gate is the NET rate-based guard (strict=False by design); GX010141 is the worst-behaved

125 clip (future over-merge target). The nightly self-flags STALE post-flip ? re-baseline once with

126 `--update-baseline`. W2 / hard-cap / R5 stay default-OFF.

127

128 **### R5 ? blob-fallback for the continuously-active blob (default-OFF, stratum tool) *(2026-06-17)***

129 The one clip the cap6+R4 milestone still fails is the ****continuously-active blob**** (mahadevpura_1:

130 ~174 s of play with no inactivity gap to split on). The W1 gap-split finds no gap; R4 finds no slow

131 tail; the result is one ~66 s mega-window that matches nothing (****tolF1 0.000****, the degenerate

132 0-match outcome [`REAL\_FOOTAGE\_VALIDATION\_REPORT.md`](REAL_FOOTAGE_VALIDATION_REPORT.md) §5 R5 names).

133

134 ****R5**** (`window\_blob\_fallback`, default-OFF) is the last resort: ****after**** the gap-split and the R4

135 trim have each had their chance, any group still longer than `window\_blob\_factor` × the cap is a

136 genuine blob ? midpoint-chop it to ? the cap, ****log the forced cut****, and ****flag every resulting**

137 window `forced\_cut=True` (surfacing the clip as one that needs rally-STATE cues ? net-crossings /

138 two-sided motion ? not a bigger cap). `blob\_fallback\_chop` in `tracknet\_runner`, wired through

139 WindowingPreset / IndexingConfig / the served path / `rally\_seg\_eval --blob-fallback`.

140

141 ****Why a factor gate, and why not just `window\_hard\_cap`? `hard\_cap` chops ****before**** R4 and fires on**

142 ****every**** gap-less over-cap window ? it over-segments normal clips (mahadevpura_2 40?68). R5 runs ****after****

143 R4, and the magnitude gate (****default 4x****) leaves a normal long rally (~12× the cap) alone, force-cutting

144 ****only**** the degenerate blob. That makes it surgical and do-no-harm.

```

145
146     **MEASURED (n=11, on the cap6+R4 milestone, R2 `tolF1`):**
147
148     | clip | milestone | + R5 | windows (milestone ? +R5) |
149     |---|---|---|---|
150     | **mahadevpura_1 (the blob)** | **0.000** | **0.238** | 2 (max 65.8 s) ? 32 (max 4.1
151     s), all `forced_cut` |
152     | mahadevpura_2 (also continuously-active) | 0.122 | 0.180 | 42 ? 100 |
153     | kushagra_singles | 0.078 | 0.110 | 45 ? 71 |
154     | Badminton_BXH_2 | 0.328 | 0.324 | 78 ? 92 (?0.004, noise) |
155     | *(the other 7 clips)* | ? | *unchanged* | no-op (no group exceeds 4× the cap) |
156
157     At factor=4, R5 touches **only** the 2 continuously-active clips + a noise-level BXH2
158     ?0.004 ? effectively
159     **do-no-harm**. It is a **STRATUM tool**: the win is the blob rescue (0.000?0.238,
160     removing the
161     mean-swinging 0-match pathology), not a corpus-mean lift (agg `tolF1` 0.323?0.354, sign
162     **3/3/5, n.s.** at
163     factor 4; the factor sweep is monotonic ? 6× is strictly do-no-harm, 3× over-chops).
164     **Default-OFF** ; turn
165     it on selectively for continuously-active footage. The R1 net over-seg guard confirms
166     it: milestone?+R5 net
167     cost 0.423?0.289, **PASS**, no merge_rate regression. [numbers:
168     `scratch/r5_evidence.py`]
169
170     ### R1 ? milestone evidence (cap=6 + R4) + the net over-seg promotion guard
171     *(2026-06-17)*
172     The R1 milestone is the smallest safe local-windowing flip: **W1 bounded windowing cap=6
173     + the R4
174     rally-END inactivity trim, no W2** (the per-clip-best config, settled by R3/R4). This
175     entry (a) re-runs
176     the full ablation ladder baseline?milestone under the **R2** `tolF1` on **n=11** with
177     honest paired
178     sign-tests, and (b) ships the **net over-seg promotion guard** that unblocks the
179     milestone.
180
181     **The full ladder (n=11, R2 `tolF1`, asymmetric ?_start=2/?_end=1.5):**
182
183     | config | tolF1 (mean [95% CI]) | tIoU-F1 | mean \[?end\] |
184     |---|---|---|---|
185     | baseline (all-OFF) | 0.091 [0.026, 0.179] | 0.195 | 52.1 s |
186     | + W1 cap=12 | 0.168 [0.100, 0.241] | 0.357 | 6.5 s |
187     | + cap=6 (R3) | 0.276 [0.174, 0.380] | 0.407 | 5.7 s |
188     | **+ R4 inact_end=1.5 (milestone)** | **0.323 [0.210, 0.433]** | **0.440** | **3.8 s**
189     |
190
191     **Paired sign-tests (the promotion evidence):**
192     - **baseline ? milestone:** ?`tolF1` **+0.232** [+0.153, +0.304], **sign 10/0/1,
193     p=0.002** (the 1 tie is the
194     degenerate blob mahadevpura_1 ? both score 0.000; excluding it, 10/0). tIoU-F1 ref:
195     +0.245, 10/0, p=0.002.
196     - **cap=12 ? cap=6** (R3 was means-only ? now confirmed): ?`tolF1` **+0.108** [+0.070,
197     +0.154], **sign 10/0/1,
198     p=0.002**. **cap=6 beats cap=12, significantly.**
199     - **cap=6 ? +R4:** ?`tolF1` **+0.047** [+0.022, +0.072], **sign 8/0/3, p=0.008**
200     (reproduces the R3/R4 entry).
201
202     **The net over-seg promotion guard** (`tolerance_metrics.over_seg_guard`, surfaced in
203     `rally_seg_eval.compare()` / `--vs`). R2 §2.3.3's *strict* rule blocks promotion if the
204     candidate raises
205     `split_count` **or** `merge_count` on **any** video ? correct for an incremental cue,
206     but it **mis-fires on a
207     structural change** (mega-windows ? bounded): splitting a mega-window necessarily lifts
208     `split_count` from ~0,
209     and the raw merge **count** is non-monotonic. The milestone trips strict on 10/11

```

```

videos, **yet it strictly
190   reduces over-merge**:
191
192   - mean **`merge_rate`** (fraction of golden rallies swallowed in a merge) **0.694 ?
0.150**, dropping on **all
193   11** videos (? ?0.545, sign **0/11**, p=0.001). The "3 videos with a higher merge
*count*" (kushagra,
194   mahadevpura_1/2) all have *lower* merge *rate* ? one mega-window hiding 40 rallies is
`count=1, rate=1.0`;
195   bounding it into pieces that each glue 2 neighbours is `count=9, rate=0.5` (**fewer**
rallies hidden).
196
197   So the guard scores over-seg on the **normalized rates** (the comparable, recall-
meaningful quantity), as one
198   weighted cost per video ? `w_merge.merge_rate + w_split.split_rate`, **merges weighted
higher** (a merge hides a
199   rally = recall loss; a split only fragments one, recall preserved). It **PASSES** iff
(1) the corpus-mean net
200   cost does not rise **and** (2) no video's `merge_rate` regresses. Measured: net cost
**1.388 ? 0.423** (? ?0.966),
201   no per-video `merge_rate` regression ? **PASS at every weight w_merge?{1,2,3}** (the
raw-count basis flips on the
202   weight ? exactly why rates are the default). The strict verdict is always reported
alongside, and reverting to it
203   is one line (`use the verdict's `strict_passes``). [numbers: `scratch/r1_evidence.py`]
204
205   **R1 verdict:** the milestone (cap=6 + R4) clears the refined (net-guard) promotion bar
? a significant `tolF1`
206   win on the full n=11 corpus with **no honest over-seg regression**. The 2-line config
flip
207   (`indexing.max_window_duration: 6`, `window_inactivity_end: 1.5`,
`inactivity_rally_thresh: 0.20`,
208   `inactivity_min_density: 0.30`) is **prepped on its own branch and left for the owner's
product-go** ? it changes
209   shipped behaviour, so it is NOT flipped here. (W2 net-crossings stays default-OFF +
retained ? no GT-backed lift.)
210
211   ### R3 (cap re-scope) + R4 (rally-END inactivity trim) ? measured on n=11 golden under
R2 *(2026-06-17)*
212   The first quality levers scored on the **R2 task-faithful metric** (tolerance-match
`tolF1`, asymmetric
213   ?_start=2/?_end=1.5; see [R2_EVAL_METRIC_DESIGN.md](R2_EVAL_METRIC_DESIGN.md)), on the
**n=11** golden corpus
214   (now incl. 5 real ground-truth clips: mahadevpura_1/_2, GX010128, GX030094,
Badminton_BXH_2).
215
216   - **R3 ? re-scope the cap (12 ? 6 s).** The 12 s cap was too coarse. Sweep (W1, no R4):
`tolF1` cap=6 **0.273**
217   > cap=8 0.234 > cap=12 0.171. So R1's recommended cap is **6 s**, not 12. (Means-based
lead across the
218   sweep; confirm with the paired sign-test before locking the config value.)
219   - **R4 ? rally-END inactivity trim** (`window_inactivity_end`, **default-OFF**). The
measured gap is the
220   rally-*end* (|?end| ? |?start| on every ground-truth clip): after a rally the shuttle
keeps moving *slowly*
221   above `velocity_thresh` (picked up / walked), so the velocity window over-extends its
END. R4 trims that
222   post-rally drift tail back to the last frame whose trailing window stays dense with
**fast** motion
223   (velocity > `inactivity_rally_thresh`); only cuts (recall can't rise), START untouched
(already ?Gemini).
224   `_trim_inactive_tail` in `tracknet_runner`, wired through WindowingPreset /
IndexingConfig / served path /
225   `rally_seg_eval --inactivity-end`.
226

```

```

227      **MEASURED (n=11, on top of W1 cap=6; sweet spot ?_end-window=1.5 s,
rally_thresh=0.20, min_density=0.30):**
228
229      | metric | cap=6 | + R4 | ? |
230      |---|---|---|---|
231      | tolF1 | 0.276 | **0.323** | **+0.047** (95% CI [+0.022,+0.072], **sign-test 8/0,
p=0.008**) |
232      | mean \|?end\| | 5.71 s | **3.81 s** | ?1.9 s (toward Gemini's ~1.0) |
233      | mean split-count | 5.5 | 5.1 | net DOWN (only mahadevpura_2 +1; GX010128 4?2, BXH2
5?4) |
234
235      **R4 beats the hard-cap mechanism for ends:** hard-cap got \|?end\| 1.07 s but split
19.5 (heavy
236      over-production); R4 gets a *significant* tolF1 lift + tighter ends with splits **net-
down** ? it cuts at
237      the real inactivity point, not a blind midpoint. Cumulative tolF1: baseline 0.096 ?
cap12 0.171 ? cap6
238      0.276 ? +R4 **0.323**. Default-OFF; promote via config once R1's net-over-seg guard +
corpus growth land.
239
240      ### Nightly rally-quality regression tripwire ? the dual-eval-set discipline made
automatic *(2026-06-17)*
241      A scheduled nightly job re-scores the **golden corpus**
(`output/human_lovo_manifest.json`, n=11) under HEAD
242      and trips a wire if anything regresses vs a frozen baseline. It is a *tripwire*, **not**
a re-training run:
243      the WASB GPU trajectories are **CACHED** and only the windowing/scorer changes night to
night, so the re-score
244      is **CPU, seconds, no GPU/Gemini** (`scripts/nightly_regression.py` ?
`rally_seg_eval.evaluate`).
245
246      - **Two tracked configs**, both re-scored every night: **DEFAULT** (the SERVED
production windowing, all
247      milestone knobs OFF ? guards shipped behaviour) and **MILESTONE** (R3 cap=6 + R4
inactivity trim ? guards the
248      candidate *before* it's flipped ON). Frozen numbers (n=11, this commit): DEFAULT tolF1
**0.091** / f1@0.5 0.194;
249      MILESTONE tolF1 **0.323** / f1@0.5 0.440 (matches the R3/R4 row above ? cross-checks
the harness).
250      - **What trips it:** any gated aggregate drop beyond tolerance (`tol_f1`, tIoU
`f1@0.5`/`f1@0.25`; `merge_rate`
251      rise) **or** any per-video **over-seg guard** rise (`split_count`/`merge_count` may
not increase on ANY video ?
252      the R2 $2.3.3 guard). Boundary-error medians + seg_ratio are outlier-dominated ?
reported, **not** gated. Exit
253      1=regression, 4=stale (corpus/config changed ? re-snapshot), writes
`output/nightly_regression/<date>.md`.
254      - **Default-ADDITIVE ? it does NOT change the promotion gate.** The baseline is tied to
the LOCAL golden corpus
255      (not committed) and is re-snapshotted (`--update-baseline`) after an intended
improvement or when the corpus
256      grows. Realises `memory/eval-dual-set-regression` (the growing GOLDEN-finalized set);
the future **RAW eval
257      set** (currently-bad videos) plugs in as a third config. Wired as a local Windows
Scheduled Task
258      (`scripts/nightly_regression.ps1` + `scripts/register_nightly_task.ps1`) ? a cloud
agent can't run it because
259      the cached trajectories live outside the repo.
260
261      ### REAL-FOOTAGE VALIDATION + first ground-truth at-par ? over-merge fix confirmed; the
gap is the rally-END *(2026-06-16)*
262      The capstone of the day: the W1/W2/W1.5 windowing work, measured on **6 real
Kushagra/Yash clips** (fully-local
263      no-AI WASB, vs Gemini as a strong reference) and ? for the first time ? against **owner-
hand-labeled ground truth**

```


264 on two of them. Full report:
 [`REAL\_FOOTAGE\_VALIDATION\_REPORT.md`](REAL_FOOTAGE_VALIDATION_REPORT.md) (adversarially
 265 verified ? a 17-agent synthesis killed 6/12 of its own draft claims). Reusable at-par
 analyzer: `scratch/at\_par.py`.

266

267 ***(1) The over-merge fix is real at scale**** (full-6, local-vs-Gemini, Gemini=reference
 not truth):

268

269 | aggregate (n=6 real clips) | baseline | W1 (cap=12) |
 270 |---|---|---|
 271 | windows (vs Gemini's 165) | 48 (mega-windows) | 250 |
 272 | mean window length | 62.9s | ****10.8s**** |
 273 | merge-groups (1 window ? 2 rallies) | 27 | ****10**** |
 274 | mean matched-IoU | 0.23 | 0.39 |

275

276 The signature pathology (GX010129: ****2 windows / 267s / 27 rallies trapped****, mIoU 0.0)
 is gone. ****But W1 over-corrects****

277 into over-segmentation (250 vs 165, ~1.5x) and it's ****clip-content-driven, NOT venue****
 (mahadevpura-0 over-splits 1.55x

278 like the GX clips ? the "GoPro-vs-court" story is unsupported at n=3/venue).

279

280 ***(2) First ground-truth at-par**** (vs human-labeled rallies; the honest scoreboard):

281

282 | clip (golden rallies) | metric | Gemini (cloud) | best LOCAL (no-AI) |
 283 |---|---|---|---|
 284 | mahadevpura-2 (40) | F1@0.5 / F1@0.25 | 0.557 / 0.835 | 0.296 / 0.722 (W1+W2+hardcap)
 |
 285 | GX010128 (52) | F1@0.5 / F1@0.25 | 0.813 / 0.857 | 0.457 / 0.756 (W1+W2) |

286

287 Local is ****~86?88% of Gemini on rally DETECTION (F1@0.25)**** but only ****~53?56% on tight
 boundaries (F1@0.5)****. On

288 GX010128 local even ****out-recalls Gemini**** ? it overlaps all 52 golden rallies; Gemini
 matched only 39/52 (missed 13).

289

290 ***(3) ? The gap is the rally-END, confirmed at n=2 ? START is already at Gemini
 parity.**** Boundary error (matched

291 pairs, median seconds):

292

293 | clip | local \\\?start\\ | Gemini \\\?start\\ | local \\\?end\\ | Gemini \\\?end\\ |
 294 |---|---|---|---|
 295 | mahadevpura-2 | 1.74 | 1.46 | ****3.12**** | 1.19 |
 296 | GX010128 | 0.78 | 0.98 | ****2.12**** | 0.76 |

297

298 Local nails the START (? or better than Gemini); the ****entire**** residual gap is the
 rally-END over-extending (the

299 velocity-windowing ends at the last ***in-flight*** frame, not the last ***sustained-active***
 one). ? ****next lever = a**

300 sustained-inactivity rally-END rule (See et al.), NOT a generic boundary head and NOT a
 serve/START classifier

301 (START is solved).**

302

303 ****Lessons (for posterity):****

304 - ****Count parity is a deceptive proxy.**** mahadevpura-2 hit 1.03x of Gemini's count yet
 F1@0.5 only 0.150 (W1+W2) ?

305 right count, wrong boundaries. Ground truth is the only honest judge; Gemini-count
 agreement misled us twice.

306 - ****Best windowing config is clip-dependent.**** Hard-cap is best on continuously-active
 clips (mahadevpura-1/-2) but

307 over-splits gap-rich clips (GX010128: W1+W2 0.457 > +hardcap 0.421). No single global
 config; hard-cap is content-adaptive.

308 - ****W2 (net-crossings) buys count by spending recall**** ? on the real clips it dropped
 ~28 windows (~all on mahadevpura-0,

309 62?43) for ****6 net-new misses**** and no ground-truth-backed gain (golden +0.019 n.s.).
****Stays default-OFF/retained, not promoted.****

310 - ****tIoU@0.5 over-penalizes this task.**** For a ~6s rally it tolerates only ~±1.9s and

```

the ~1.5s service window nearly
311     exhausts that ? it hides local's recall + START accuracy, and even flips config
rankings. The eval metric is now a
312     bottleneck ? **R2** (next): a **±2s tolerance-match F1 (?_start?2s, ?_end?1.5s) +
Hungarian 1:1 + an over-segmentation
313     guard + start/end boundary-error split**; tIoU/mIoU demoted to secondary.
314
315     **Regression discipline adopted (owner directive 2026-06-16):** maintain a **RAW eval
set** (currently-bad videos,
316     random-sampled) + a **GROWING GOLDEN-finalized eval set** (a video is promoted
raw?golden when an iteration brings it
317     *to par*) + a separate golden TRAINING set. **Every quality launch must not regress on
EITHER eval set** ? so a late win
318     can't silently undo an earlier one. The already-built ablation layer
(`backend/eval/ablation.py`) is the A/B mechanism;
319     wire the per-set aggregate into `run_regression.py` when R2 lands. (Privacy-safe, video-
free shipping of these sets =
320     the golden-regression-fixtures track.)
321
322     **Corpus: golden grew 6 ? 8** this session (`mahadevpura_2` 40 rallies, `GX010128` 52
rallies ? both real multicourt,
323     frame-aligned, added to `human_lovo_manifest.json`). **n=8 milestone numbers (LOVO,
tIoU-F1):** baseline 0.232 ? W1
324     0.403 ? W1+W2 0.419 ? **W1+W2+hardcap 0.437** (merge_rate 0.634 ? 0.000); milestone ?
baseline?W1+W2 = **+0.187 F1@0.5**.
325     All flags **default-OFF**; promotion to a shipped milestone is owner-gated and re-run at
the ~16-clip corpus.
326
327     ### Tier 1 (W1.5) ? hard-cap fallback: enforce the cap on continuously-active
trajectories *(2026-06-16)*
328     The real-footage 6-video local-vs-Gemini validation surfaced W1's one structural blind
spot: its
329     largest-internal-gap splitter **can't cap a window when the shuttle never stops moving**
(no inactivity
330     gap to cut at). On the real `mahadevpura-1` proxy, W1 left the 174 s blob as **2 windows
of ~87?102 s** ?
331     unwatchable, and 0% segmented vs Gemini's 9 rallies. **Fix (`window_hard_cap`, default-
OFF):** when an
332     over-long window has no qualifying internal gap, recursively chop it at its temporal
**midpoint** until
333     every piece ? `max_window_duration`. Still only cuts (never merges ? recall can't drop).
Threaded through
334     `trajectory_to_action_windows` ? `_split_overlong_groups(hard_cap=)`,
`WindowingPreset.hard_cap`,
335     `IndexingConfig.window_hard_cap`, served `trajectory_hybrid`, and `rally_seg_eval
--hard-cap/--vs-hard-cap`.
336
337     **MEASURED ? golden n=6 (W1 cap=12 ? +hard-cap):** ?F1 **+0.006** [?0.019, +0.033], **CI
spans 0,
338     sign-test 2/2, p=1.000 ? NOT significant** (+ seg_ratio +0.123: it adds some over-
split). **Honest read:
339     neutral on the clean golden set** ? the golden videos have few continuously-active >cap
windows, so the
340     fallback rarely fires. [[decision-rigor-norm]]: **not promotable on the golden number**
(CI spans 0).
341     **Real-footage value (the reason it exists), `mahadevpura-1` cached trajectory:**
342
343     | windowing | n windows | max window | mean window |
344     |---|---|---|---|
345     | baseline | 1 | 174.3s | 174.3s |
346     | W1 (cap=12) | 2 | 101.9s | 86.9s |
347     | **W1 + hard-cap** | **24** | **9.0s** | **7.2s** (? Gemini 5.7s) |
348
349     So the cap becomes a TRUE upper bound and the worst-case clip is finally segmented (24
windows vs Gemini's

```

```

350     9 ? it **over-segments**, since a blind midpoint chop is not rally-boundary-aware;
that's the honest
351     tradeoff and the motivation for the **boundary head / rally-state-aware split** (M1/R1),
the next lever).
352     **Status: retained default-OFF**, justified by the real-footage fix + the foundational
argument that
353     `max_window_duration` should be a true cap ? NOT by a golden lift (there is none at
n=6). Promote/flip with
354     more golden footage that actually contains continuous-motion blobs. Reproduce:
355     `rally_seg_eval --preset served --max-window-duration 12 --vs served --vs-hard-cap`.
Ties [[weak-signals-retained]].
356
357     ### Tier 1 (W2) ? "never-empty" safeguard makes the net-crossings gate production-safe
*(2026-06-16)*
358     Hardening that closes the one real-footage failure mode of W2 before the milestone flip.
**Found on real
359     footage:** on a noisy 4K-proxy trajectory (`mahadevpura-1`), W1 produced 2 windows but
the net-crossings
360     gate (mc=1) kept **0** ? the auto-estimated net axis + jittery shuttle track meant
neither piece registered
361     a clean crossing ? so the whole 174 s clip yielded **0 rallies (no reel)**. A gate that
can *zero out a
362     non-empty video* is not safe to layer onto the substrate. **Fix:** both gate sites
363     (`FusionSegmenter._select_for_handoff`, served; `rally_seg_eval.windows_for`, eval) now
fall back to the
364     **ungated** set whenever gating would drop *every* window of a video that had ?1. The
gate is therefore
365     strictly **pruning** ? it can shrink a candidate set but never zero it ? so W2-on-W1 is
**do-no-harm**: it
366     can only *match or beat* W1 per video, never strand a clip with no highlights.
**Verified:** golden delta
367     **unchanged** (W1 0.439 ? W1+W2 0.457, **+0.019**, 6/0, p=0.031 ? no golden video is
emptied at mc=1, so the
368     guard is a no-op there); on the real `mahadevpura-1` trajectory the guard turns the
gate's `0` back into
369     W1's 2 windows (`raw gate mc=1 = 0 kept ? guarded = 2`). 2 new tests
370     (`test_gate_a_never_empties_a_video`, `windows_for` mc=999 ? ungated). This makes
**W1+W2 the safe milestone
371     candidate** to flip on together.
372
373     ### Tier 1 (W2) ? net-crossings rally-state gate on W1: ?F1 +0.019, 6/0 *(2026-06-16)*
374     The second Tier-1 increment, on top of W1. **Hypothesis (worth re-testing):** net-
crossings (Gate-A) was a *null/negative*
375     on the OLD coarse served windowing (it had nothing to clean ? the #151 revert), but W1's
bounded windowing produces finer
376     windows with over-split fragments (seg_ratio 1.44) ? exactly what a "did the shuttle
actually cross the net?" gate should
377     remove. **Confirmed.** **Shipped:** extended `rally_seg_eval` with an optional
`min_crossings` net-crossings gate
378     (`rally_gate.gate_windows`) + a `--max-window-duration` CLI override, so the harness
measures "windowing + rally-state
379     gate". The gate itself is **existing code** (`FusionSegmenter.min_crossings`); served
path gets W1+W2 via config
380     (`indexing.max_window_duration: 12` on `fusion_hybrid` + `indexing.fusion.min_crossings:
1`) ? no new detector code.
381
382     **MEASURED on golden (n=6, LOV0; W1 cap=12, then + net-crossings ?1):**
383
384     | metric | W1 | W1 + Gate-A (mc=1) | ? |
385     |---|---|---|---|
386     | tIoU-F1@0.5 | 0.439 | **0.457** | **+0.019** (95% CI [+0.011,+0.027], **sign-test
6/0**, p=0.031) |
387     | F1@0.25 | 0.772 | 0.805 | +0.033 (6/0) |
388     | segment_count_ratio | 1.44 | 1.33 | over-split trimmed |
389

```

390 Small but ****statistically robust**** (every video improves) and ****served-measured**** ? and a satisfying reversal of the

391 Gate-A null: the cue only pays off **once the windowing is bounded** (signal value is windowing-dependent ? a generalisation

392 of the eval?serve lesson). ``mc=2`` ? neutral, ``mc=3`` hurts (trims real rallies) ? recommend ****mc=1****. Reproduce:

393 ``python -m backend.eval.rally_seg_eval --preset served --max-window-duration 12 --vs served --vs-min-crossings 1``.

394 ****Cumulative project number (golden n=6 aggregate tIoU-F1@0.5): 0.300 ? 0.439 (W1) ? 0.457 (W2) = +0.157 total.****

395 Default-OFF; promote via config. The richer multi-feature scorer (person/audio + direction-reversal/peak) is the W2

396 follow-on (needs GPU person/audio feature re-extraction on the golden videos ? now feasible with the GPU grant).

397

398 **### Tier 1 (W1) ? bounded windowing fixes the over-merge: ?F1 +0.139, 6/0 *(2026-06-16)***

399 The first quality **improvement** of the rally-quality project

(RALLY_QUALITY_RESEARCH.md §6/§7,

400 building on the Tier-0 baseline below). ****Hypothesis:**** the local segmenter doesn't miss rallies, it **fails to segment** ?

401 "shuttle is moving" is treated as "rally in progress," so dead time + serves bridge rallies into mega-windows (Tier-0

402 merge_rate 0.523; kushagra_singles = 2 windows for 32 rallies). ****Shipped (default-OFF, config-gated):****

403 ``trajectory_to_action_windows`` gained ``max_window_duration`` + ``min_split_gap`` ? any window longer than the cap is

404 recursively split at its largest internal inactivity gap (? ``min_split_gap``; the most likely rally boundary, the See-et-al.

405 sustained-inactivity spirit). It only ****cuts****, never merges/extends, so recall cannot drop. Wired through ``WindowingPreset``/

406 ``build_windows`` (eval) and ``IndexingConfig.max_window_duration`` ? ``trajectory_hybrid.process_video`` (served), default 0=OFF.

407

408 ****MEASURED on golden (n=6, LOV0, ``rally_seg_eval --vs``, recommended cap=12s / min_split_gap=0.5):****

409

metric	baseline (Tier 0)	W1 (cap=12)	?
tIoU-F1@0.5	0.300	**0.439**	**+0.139** (95% CI clears 0; **sign-test 6/0** , p=0.031)
F1@0.25	0.501	**0.772**	+0.271
merge_rate	0.523	**0.038**	?0.485 (over-merge essentially eliminated)
segment_count_ratio	0.700	1.44	now mildly over- <i>*splits*</i> ? W2's scorer to clean up

416

417 ****Robust, not overfit:**** every cap in 6?20 s clears 0 with 6/0 sign-test (the win isn't a knife-edge); smaller caps trade

418 more over-split for more F1 (cap=6 ? +0.194 but seg_ratio 1.8). Picked cap=12 (rec-rally upper bound) as the recommended

419 default; full sweep in the PR. ****Eval?serve:** this IS the served windowing + the cap, so the win is served-measured ? no

420 proposal?served transfer risk****** (unlike Gate-A). ****Cumulative project number (golden n=6 aggregate tIoU-F1@0.5):**

421 0.300 ? 0.439 (+0.139).****** (An exact "% of the gap to Gemini closed" needs Gemini measured on the **same** golden set +

422 metric ? a TODO; the existing 0.93-clean / 0.66-multi-court Gemini refs are per-venue, not this aggregate, so don't mix

423 them.) Kept ****default-OFF**** pending owner go to flip the served default (one config line); the residual over-split is the

424 explicit hand-off to ****W2**** (rally-state features into the scorer).

425

426 **### Tier 0 (E1) ? over-segmentation-aware eval harness + frozen local baseline *(2026-06-16)***

427 First landed work of the ****rally-quality project****

(RALLY_QUALITY_RESEARCH.md §7 ?

```

428     "bridge the local?Gemini gap"). **Why:** plain temporal-IoU F1 is nearly blind to the
*over-merge* the local
429     segmenter exhibits (one window swallowing many rallies), so we couldn't *measure* the
failure we set out to fix.
430     **Shipped:** `segmentation_metrics.merge_split_report` ? a bipartite overlap-graph
breakdown into
431     matched / **merge** (1 pred ? ?2 rallies) / split / missed / spurious +
`merge_rate`/`split_rate`, the
432     over-segmentation metric for this *binary* setting (the classic single-class edit score
is degenerate here ?
433     rally/background strictly alternate, so it collapses to the segment-count difference ?
and stays omitted);
434     plus `backend/eval/rally_seg_eval.py`, the one-command eval: golden manifest +
golden_features `gts` ? window
435     each cached trajectory under a `WindowingPreset` ? score (tIoU-F1 + F1@k + segment-count
+ merge/split,
436     bootstrap CI), and `--vs` diffs two windowings with a paired sign-test (the per-tier ?).
Pure/offline (CPU);
437     18 new tests; suite **722 green**, ruff + mypy clean.
438
439     **FROZEN BASELINE ? current local heuristic** (SERVED windowing 0.02/2.0/2.0, windows-
as-rallies, no AI; n=6 golden, IoU 0.5):
440
441     | metric | mean [95% CI] |
442     |---|---|
443     | tIoU-F1@0.5 | **0.300** [0.119, 0.494] |
444     | F1@0.25 | 0.501 [0.239, 0.747] |
445     | **merge_rate** | **0.523** [0.258, 0.788] |
446     | segment_count_ratio | 0.700 [0.405, 0.957] |
447
448     The `0.300` **matches the independent served-side measurement below** (min_crossings=0 =
`wasb_hybrid` = 0.300) ? cross-validated.
449     **Tier-0 "number" = this baseline** (Tier 0 establishes the ruler; it does not change
quality). The over-merge is now a metric:
450     **merge_rate 0.523** ? 52% of golden rallies swallowed into a merge group
(kushagra_singles = 1.00, just **2 windows for 32 rallies**;
451     gbaaddy 0.85; mahadevpura 0.52). Every later tier's delta is measured against this via
`python -m backend.eval.rally_seg_eval --vs` ?;
452     per-tier + cumulative deltas land here.
453
454     ### Served-side Gate-A measurement on the LIVE FusionSegmenter ? the independent run
behind the #151 revert *(2026-06-14)*
455     The owner-gated follow-up flagged by the (now **REVERTED**) SERVED-DEFAULT entry below.
An **independent** served-side check ?
456     separate from, and corroborating, the [#148](https://github.com/avidullu/badminton-
highlight-indexer/pull/148) litmus + #151's
457     run ? of whether the Gate-A default flip (PR #146) regresses the live path. **Verdict:
it does ? the flip was reverted in
458     [#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151); this entry is
the detailed measurement.** Driven the
459     **real** served seams (`_enrich_candidate_stats` ? `net_crossings`,
`_select_for_handoff` ? the gate) over each golden video's
460     **cached** WASB trajectory, at the **production windowing** `process_video` actually
uses
461     (`velocity_thresh=0.02, merge_gap=2.0, min_window=2.0`) ? no WASB re-run (the runner has
no cache-hit early-return), no GPU, no
462     Gemini. Config loaded exactly as production (`load_config()`); segmenters from the
registry. Repro:
463     `scratch/served_gateA_verify.py` + `scratch/served_gateA_sweep.py` (artifacts in
`output/gateA_served_verify/`); the standing
464     productionized guard is `backend/eval/served_gate_a.py` (#148).
465
466     - ** (a) No-regression invariant CONFIRMED on real served code.** At `min_crossings = 0`,
`fusion_hybrid`'s seeding **and**
467     handoff set are **byte-identical to `wasb_hybrid`** on all 6 golden videos (same

```

candidate windows; handoff = all windows =

468 the base seam). The "fusion adds nothing when the gates are off" property is now
verified on the *live* served path, not

469 only by code review. ?

470 - ****(b) The +0.074 win does NOT transfer to the served windowing.**** The offline
promotion (+0.074, 6/6, p=0.031) was measured

471 on the eval harness's ****recall-oriented *proposal* windowing**** `(0.005, 1.0, 0.5)`,
where CONTROL ****over-segments****

472 (segment-count ratio 1.68) and Gate-A's job is to clean up that fragmentation (?
1.01). The ****served path uses a coarse**

473 **windowing**** `(0.02, 2.0, 2.0)` that already merges aggressively (low over-
segmentation), so Gate-A has little FP to remove

474 and instead trims ****real**** rallies whose WASB trajectory registered only 0?1 net
crossings. ****Candidate-handoff-level F1**

475 (pre-Gemini, IoU ? 0.5) on the served windowing, n=6:******

476

477 | min_crossings | mean F1 | precision | recall | real-rally windows lost vs off | ?F1
vs off |

478 |---|---|---|---|---|---|

479 | 0 (Gate-A OFF = `wasb\_hybrid`) | 0.300 | 0.317 | 0.290 | 0 | ? |

480 | 1 | ****0.303**** | 0.327 | 0.288 | 1 | ****+0.003**** |

481 | ****2 (shipped default, PR #146)**** | 0.292 | 0.337 | 0.261 | ****12**** | ****?0.008**** |

482 | 3 | 0.269 | 0.338 | 0.226 | 28 | ?0.031 |

483

484 At the shipped `mc=2`, mean F1 goes ****0.300 ? 0.292 (?0.008)**** and Gate-A drops ****12**
windows that match golden rallies******

485 (net_crossings almost all exactly 1.0), concentrated in doubles (`targetest\_doubles`
?0.051). All deltas are ****tiny / within**

486 n=6 noise******, so this is not a catastrophic regression ? but `mc=2` is ****not**** a
served-side ****improvement****, and the

487 held-shuttle "guard" is removing genuine rallies, not just feeds.

488 - ****Served relevance (honest).**** A dropped candidate window ****never reaches Gemini****, so
each lost real rally is an

489 ****unrecoverable served recall loss****; the precision "gain", by contrast, is partly
what Gemini already does at the handoff ?

490 so the served-side tradeoff is at best neutral and plausibly net-negative, i.e. the
candidate-level ?0.008 likely

491 ****understates**** the served downside.

492 - ****Root cause = eval ? serve windowing.**** The harness promotes/refuses variants on a
windowing the production path does not

493 serve. Gate-A's value is conditional on over-segmentation; the coarse served windowing
doesn't over-segment, so the gate

494 inverts. [[decision-rigor-norm]]: the foundational issue (harness windowing ? served
windowing) outweighs the single

495 proposal-windowing number.

496

497 ****Outcome ? REVERTED (see the entry below).****

[#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151)

498 restored the pre-#146 state: `config.json` `indexing.default\_segmenter` ? `gemini` (the
`indexing.fusion` block removed) and

499 `FusionConfig.min\_crossings` default ****2 ? 0**** ? Gate A is now ****OFF** by default but
still opt-in******

500 (`indexing.fusion.min\_crossings: 2`), the persisted-superset invariant intact. Our
independent numbers match #151's (mean

501 served ?F1 ?0.008, ~12 rallies dropped, 2/6 losses). If Gate A is ever opted back in on
the served path, `mc=1` is the

502 least-bad operating point (?flat F1, 1 rally lost). ****Open follow-up:**** ****align the**
experiment-harness windowing with the

503 served windowing****** (or re-run promotions on the production windowing) so a harness lift
actually predicts served behaviour ?

504 the real lesson here. The offline ablation + unit tests remain valid for what they
measured.

505

506 **### PERSONALIZATION Phase-0 quality checkpoint ? the eval/safety RAILS for the per-user
feature *(2026-06-14)***

```

507     The owner adopted the **HYBRID** posture: personalization as a **flag-gated FEATURE on a
generalization-first baseline**,
508     paired with a **contribution flywheel** so per-user gains and the shared baseline
compound together (full design in
509     [PERSONALIZATION_PLAN.md](PERSONALIZATION_PLAN.md), merged
[#144](https://github.com/avidullu/badminton-highlight-indexer/pull/144)).
510     This checkpoint landed the three **honest-eval RAILS** that keep a per-user/small-N
tuning layer from promoting noise or
511     silently regressing a user:
512
513     - **Per-user promotion gate ? [#141](https://github.com/avidullu/badminton-highlight-
indexer/pull/141)**
514         ( `backend/eval/per_user_gate.py` ): a **min-N floor** + **structural-guard exemption**
? per-user/small-N tuning can't
515         promote noise and can't disable a structural guard (the proven Gate-A held-shuttle
guard stays on regardless of a thin
516         user corpus).
517     - **Held-out-USER learning curve ? [#142](https://github.com/avidullu/badminton-
highlight-indexer/pull/142)**
518         ( `backend/eval/per_user_eval.py` ): turns the vague "<K videos to superior quality"
claim into a **falsifiable release
519         number** (the `smallest_k_with_lift` measured on held-out users).
520     - **Per-user no-regression guard ? [#143](https://github.com/avidullu/badminton-
highlight-indexer/pull/143)**: a
521         retrain/baseline-upgrade can **never make a user worse than their incumbent** (frozen
per-user CONTROL; a regressing
522         candidate is refused).
523
524     **Read HONESTLY:** these are the **eval/safety rails** for the personalization feature ?
**not** new measured
525     rally-detection wins. No F1 moved this checkpoint; what landed is the discipline that
lets per-user tuning ship without
526     regressing the baseline or over-claiming on n=6-per-user data.
527
528     **Locked owner decisions (2026-06-14):** identity = *design for the north-star (multi-
user/cloud), implement for current
529     use (single-user local)*; the inline per-user classifier sits **behind a cloud-load
interface** (local store = one impl);
530     **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate** (a one-
video user is ALWAYS fully segmented by
531     the baseline batteries); **free-tier videos ARE pooled** into the shared baseline
(consent baked into free terms,
532     minors-exclusion enforced at the pooling query); and the previously-pending structural-
guard sub-decision is now LOCKED ?
533     **Gate-A-only structural + `min_n = 5`**.
534
535     ### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio =
promising (not promotable) *(2026-06-14)*
536     The GPU person-feature extraction finished all 6 golden videos ( `fusion_golden.py` ?
`output/*_golden_features.json`, the
537     replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats
(C1 ? bootstrap 95% CI + paired sign
538     test; never a bare mean at n=6)**:
539
540     | Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears
0 |
541     | ---|---|---|---|---|---|
542     | shuttle-only | 0.307 | ? | ? | ? | ? |
543     | + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) |
**No** |
544     | + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L
(p=0.688) | **No** |
545
546     **Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign
tests n.s.) at n=6 ? **both stay

```

547 default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ?
 `players\_in\_court` counts spectators);
 548 per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising
 lead ? +0.079 / +27.7%, wins 4/6
 549 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above**
 shuttle-only (0.307) ? but n=6 can't
 550 confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and
 measure audio on shuttle-only
 551 *directly* (without the person handicap). Recorded in
 `eval\_baselines/experiment\_leaderboard.json`. **Negative/uncertain
 552 results are kept on purpose ? this is the honest ablation backbone for the paper aim.**
 553
 554 ### Gate-A served-default flip ? **REVERTED (eval?serve regression)** *(2026-06-14)*
 555 **The #146 flip of Gate-A (net-crossings ? 2) to the live served default was REVERTED.**
 A served-side check ?
 556 the repeatable litmus in [#148](https://github.com/avidullu/badminton-highlight-
 indexer/pull/148)
 557 (`backend/eval/served\_gate\_a.py` + `tests/test\_served\_gate\_a\_regression.py`),
 corroborated by an independent
 558 served run ? found Gate A is **not a win on the served path**: on the production
 (COARSE) windowing it is
 559 **neutral-to-negative, mean served ?F1 ?0.008**, dropping ~12 real rallies across the
 golden 6 (2 losses, 0
 560 clear wins). It is a **confirmed served regression**, so the owner directed a revert to
 restore the known-good
 561 state.
 562
 563 **Why eval ? serve (the honest root cause).** The +0.074 below is a property of the
 **offline harness's
 564 permissive PROPOSAL windowing** (`velocity_threshold=0.005, merge_gap=1.0,
 min_window=0.5`), which emits many
 565 short candidate windows ? leaving abundant held-shuttle junk for Gate A to cut. The
 served COARSE windowing
 566 (`velocity\_threshold=0.02, merge\_gap=2.0, min\_window=2.0`) already merges flight bursts
 into a few long windows
 567 and drops short ones (served over-seg ratio already **< 1**, i.e. under-segmentation),
 so there is little junk
 568 left to cut and the gate instead occasionally trims a borderline true window ? slightly
 negative ?F1. The gate
 569 *itself* is correct (#148's control test reproduced +0.074 / 6-of-6 / p=0.031 exactly
 under proposal
 570 windowing **on the original-6 corpus** ? archived; see the re-grounding note below); the
 **operating point is
 571 what differs**. This is the canonical eval?serve trap.
 572
 573 **What the revert restores** (back to the pre-#146 no-regression state):
 574 - `config.json` ? `indexing.default\_segmenter` **`fusion\_hybrid` ? `gemini`** (the prior
 served detector); the
 575 explicit `indexing.fusion` block added by #146 is removed.
 576 - `backend/config/models.py` ? `FusionConfig.min\_crossings` default **2 ? 0** (Gate A
 OFF by default).
 577 - Tests (`tests/test\_config.py`, `tests/test\_fusion\_hybrid.py`) restored to the Gate-A-
 OFF defaults; the
 578 **persisted-superset invariant is preserved** (gating only ever filtered the AI
 handoff, never persistence).
 579 - `eval\_baselines/experiment\_leaderboard.json` ? the #146 `promotion` record carries a
 `reverted (eval?serve)`
 580 note.
 581
 582 **Gate A is NOT deleted ? it remains opt-in** (`indexing.fusion.min\_crossings: 2`) for
 footage with more
 583 held-shuttle/service-feed false positives than the golden 6, and the offline ablation
 evidence below stands
 584 (it is a real proposal-windowing win). The path back to serving it is to **align the
 served windowing toward

585 the proposal operating point** and re-run the #148 litmus to confirm the lift transfers.
 The #148 litmus is
 586 now the standing guard: it pins the honest measured behaviour on the **present golden
 corpus** and trips if a
 587 future windowing/corpus change silently shifts it.
 588
 589 > **Re-grounded 2026-06-18.** The original-6 single-court golden clips this litmus was
 calibrated on have been
 590 > retired from the box (their labels are gone); the present golden set is **9 multicourt
 clips**. On that
 591 > recall-starved corpus (WASB tracks one shuttle while several courts rally ? served
 over-seg ratio < 1) the
 592 > served Gate-A ?F1 is now **?0.034** (was ?0.008), and under the permissive proposal
 windowing the gate is
 593 > **~F1-neutral** (?F1 ? ?0.001) while still cutting over-seg (1.16?0.64) ? it no longer
 reproduces the +0.074
 594 > live. The canonical **+0.074 / 6-of-6 / p=0.031** win remains a real result of the
 original-6 corpus,
 595 > preserved in the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-
 experiment/README.md).
 596
 597 ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
 corroboration *(2026-06-14)*
 598 **Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The
 measured win
 599 (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg
 1.68?1.01 ? the
 600 [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md))
 clears the honest

7. assistant (2026-06-19T15:52:19.668Z)

Now I have the two stale_text passages. Let me verify both verbatim matches and check the surrounding context. Let me look at the manifest entries to confirm what's actually in there.

8. user (2026-06-19T15:52:21.350Z)

adarsh_avi_singles
 kushagra_singles
 largetest_doubles
 gbaaddy
 testlarge_short
 mahadevpura_singles
 mahadevpura_2
 GX010128
 mahadevpura_1
 GX030094
 Badminton_BXH_2
 GX010141
 GX010137
 Boxhill_Doubles
 GX020094

9. assistant (2026-06-19T15:52:25.709Z)

The manifest has 15 entries, and notably it INCLUDES all 6 original clips (adarsh_avi_singles through mahadevpura_singles). This directly contradicts the re-grounding note's claim that the original-6 "labels are gone" and the set is "9 multicourt clips." Let me verify the exact stale_text strings appear verbatim in the file, and check the P7 stratified table is consistent (line 66-68 says 11 single + 4 multicourt = 15).

Let me grep for both stale_text fragments to confirm uniqueness and exact match.

10. user (2026-06-19T15:52:27.243Z)

18:**Golden corpus = the 6 labelled matches** in `output/human\_lovo\_manifest.json` (owner-confirmed)

11. user (2026-06-19T15:52:28.494Z)

590:> retired from the box (their labels are gone); the present golden set is **9 multicourt clips**. On that

12. assistant (2026-06-19T15:52:39.897Z)

Both stale_text strings match verbatim and are unique. Now let me verify the key factual claims against the actual data.

****Fix 1 verification:****

- The doc (line 18-20) says "Golden corpus = the 6 labelled matches" and enumerates the original 6.
- The manifest has 15 entries, including all 6 of those originals.
- The doc's own later entries cite "n=15 golden" (line 62, 68) and "n=13" (line 104, 109).
- So the present-tense definitional claim "the 6 labelled matches" is genuinely stale.

****Fix 2 verification:****

- The re-grounding note (line 589-595) claims the original-6 "labels are gone" and "the present golden set is 9 multicourt clips."
- But the manifest STILL contains the original 6 (adarsh_avi_singles, kushagra_singles, etc.) and has 15 total entries.
- The P7 table (lines 66-68) confirms 11 single-court + 4 multicourt = 15 ? NOT all-multicourt.

This is a more nuanced case. The proposed corrected_text re-casts "the present golden set is 9 multicourt clips" ? "at the time of this re-grounding the golden set was 9 multicourt clips (since grown to 15)". Let me carefully examine whether the measurement provenance is preserved and whether the correction is accurate. Let me re-read the full re-grounding note and the surrounding commits.

13. user (2026-06-19T15:52:39.960Z)

582 **Gate A is NOT deleted ? it remains opt-in** (`indexing.fusion.min\_crossings: 2`) for
footage with more
583 held-shuttle/service-feed false positives than the golden 6, and the offline ablation
evidence below stands
584 (it is a real proposal-windowing win). The path back to serving it is to **align the
served windowing toward
585 the proposal operating point** and re-run the #148 litmus to confirm the lift transfers.
The #148 litmus is
586 now the standing guard: it pins the honest measured behaviour on the **present golden
corpus** and trips if a
587 future windowing/corpus change silently shifts it.
588
589 > **Re-grounded 2026-06-18.** The original-6 single-court golden clips this litmus was
calibrated on have been
590 > retired from the box (their labels are gone); the present golden set is **9 multicourt
clips**. On that
591 > recall-starved corpus (WASB tracks one shuttle while several courts rally ? served
over-seg ratio < 1) the
592 > served Gate-A ?F1 is now **?0.034** (was ?0.008), and under the permissive proposal
windowing the gate is
593 > **~F1-neutral** (?F1 ? ?0.001) while still cutting over-seg (1.16?0.64) ? it no longer
reproduces the +0.074
594 > live. The canonical **+0.074 / 6-of-6 / p=0.031** win remains a real result of the
original-6 corpus,
595 > preserved in the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-
experiment/README.md).
596
597 ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*

14. assistant (2026-06-19T15:52:47.062Z)

This re-grounding note was added 2026-06-18 (per recent commit `e59b2fd test(eval): re-ground served Gate-A litmus on the present (multicourt) golden set`). There's a real tension here. The note claims "the original-6 single-court golden clips ... have been retired from the box (their labels are gone)". But the manifest STILL contains all 6 originals. Let me check whether the labels are actually still present ? i.e. is the manifest the source of golden labels, or is there a separate label store that may have been retired? Let me check the served_gate_a litmus and its test, plus the manifest structure.

15. user (2026-06-19T15:52:47.631Z)

```
keys of first entry: ['name', 'trajectory', 'fps', 'frame_width', 'labels']
{'fps': 59.94,
 'frame_width': 1920,
 'labels': 'C:/Users/avidu/Projects/Annotation Setup/Collect/Golden '
           'Labelled/Adarsh and Avi.rallies.csv',
 'name': 'adarsh_avi_singles',
 'trajectory': 'C:/Users/avidu/Projects/Annotation '
              'Setup/Collect/Trajectories/adarsh_avi_full_traj.csv'}
```

16. user (2026-06-19T15:52:47.795Z)

```
backend\eval\rally_seg_eval.py:38:#: SHORT (motion blips, background-court fragments on
multicourt footage); filtering to long rallies
backend\eval\rally_seg_eval.py:47:#: of the single-vs-multicourt strata, kept in sync with
``docs/GOLDEN_VIDEOS.md`` (the clips tagged
backend\eval\rally_seg_eval.py:48:#: "multicourt"). Multicourt footage carries background games
on adjacent courts ? the dominant
backend\eval\rally_seg_eval.py:51:MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128",
"Badminton_BXH_2", "Boxhill_Doubles"})
backend\eval\rally_seg_eval.py:55:      """"'multicourt'`` or ``'single'`` for a golden video.
Honors an explicit manifest
backend\eval\rally_seg_eval.py:56:      ``court_type`` field when present, else falls back to
membership in ``MULTICOURT_CLIPS``
backend\eval\rally_seg_eval.py:61:      return "multicourt" if video.get("name") in
MULTICOURT_CLIPS else "single"
backend\eval\rally_seg_eval.py:272:      """"The long-rally lens payoff: ``{all vs long(>long_tau)}
× {single vs multicourt}``` aggregates.
backend\eval\rally_seg_eval.py:275:      background-court fragments on multicourt footage).
Filtering to long rallies should strip those
backend\eval\rally_seg_eval.py:276:      FPs ? precision (``tol_precision``) should jump, most on
multicourt. This table is how we read
backend\eval\rally_seg_eval.py:286:      courts = ("single", "multicourt", "all")
backend\eval\rally_seg_eval.py:368:      """"Render the long-rally stratified table: {all vs long}
× {single, multicourt, all-courts}.
backend\eval\rally_seg_eval.py:371:      short false positives, precision rises (most on
multicourt). ``n`` is the videos with >=1 golden
backend\eval\rally_seg_eval.py:485:      "{single vs multicourt} aggregates
(court type from GOLDEN_VIDEOS.md / the "
```

17. assistant (2026-06-19T15:52:54.883Z)

This is decisive. The current code in `backend/eval/rally\_seg\_eval.py:51` defines:

```
```python
MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2",
"Boxhill_Doubles"})
```
```

Only ****4 multicourt clips****. The manifest has 15 entries, and the original-6 clips (adarsh_avi_singles etc.) are STILL in the manifest WITH their labels (the first entry has `labels` pointing to "Adarsh and Avi.rallies.csv"). So:

1. The original-6 labels are NOT gone ? they are still in the manifest with valid label paths.
2. The present golden set is NOT "9 multicourt clips" ? it is 15 clips, of which only 4 are multicourt and 11 are single-court.

So the re-grounding note's factual claims ("original-6 labels are gone", "the present golden set is 9 multicourt clips") are genuinely contradicted by the current code/data.

However, the proposed Fix 2 correction is problematic. The re-grounding note attributes the ****?0.034 measurement**** to the n=9 multicourt set. If the manifest now has 15 clips (11 single + 4 multi), the ?0.034 figure was presumably measured on a different corpus state than what currently exists. Let me check the served_gate_a litmus test to understand what corpus the ?0.034 was measured on, and whether the n=9 claim ever matched reality.

18. user (2026-06-19T15:52:57.157Z)

```
d749fb9 test(eval): re-ground served Gate-A litmus on the present (multicourt) golden set (#236)
20f8aa3 feat(eval): P7 long-rally (>5s) quality lens + reel knob (both default-OFF) (#220)
67b6927 feat(config): R1 milestone ? flip windowing defaults ON (cap=6 + R4), re-verified at
n=13 (#204)
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
```

19. user (2026-06-19T15:52:58.127Z)

```
commit e59b2fda6c6c4c408f9e12e66ea6a286057cb0bc
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Thu Jun 18 15:58:14 2026 +0530
```

test(eval): re-ground served Gate-A litmus on the present (multicourt) golden set

The served Gate-A regression litmus was calibrated on the original 6 single-court golden clips (2026-06-14). Those clips have been retired from the box (their labels are gone); the present golden set is 9 multicourt clips. Three assertions were stale and failed the local full-suite pre-LGTM gate (they SKIP in CI ? cv2-gated ? so never go red there):

- runs-on-all: assert len(results) == len(_SPECS) and >= 6 (was a hardcoded 6).
- no-material-regression: re-measured served mean ?F1 ? -0.034 (was ? -0.008); band re-measured to admit the recall-starved multicourt loss while still tripping on a material regression.
- proposal-windowing: re-measured ?F1 ? -0.001 / over-seg 1.16?0.64 (was +0.074 / 1.68?1.01). Now pins the durable direction-of-effect (the gate cuts over-seg) plus the present ~F1-neutral snapshot. The canonical +0.074 / 6-of-6 / p=0.031 win is a property of the retired original-6 corpus (preserved in docs/archives) and is no longer reproduced here.

Re-grounded the module narrative to the present multicourt corpus. The pinned magnitudes are a date-stamped snapshot and will need re-measurement as the golden set grows.

Verified on the local golden files: 5/5 served-litmus tests pass; full suite green (864 passed, cov 81.23%); ruff + mypy clean.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

20. assistant (2026-06-19T15:53:03.216Z)

This is crucial context. The re-grounding commit (e59b2fd, 2026-06-18) was based on a golden set state of "9 multicourt clips" at that time. But the manifest has since grown to 15 (the most recent commits added Boxhill clips), and critically, the manifest CURRENTLY contains all 6 originals WITH their labels. Let me check the served_gate_a litmus test to see what it actually asserts now, to verify whether the "9 multicourt clips" / "labels are gone" claim matches the present code.

21. user (2026-06-19T15:53:05.299Z)

- no-material-regression: re-measured served mean $F1$? -0.034 (was ? -0.008); band re-measured to admit the recall-starved multicourt loss while still tripping on a material regression.
- proposal-windowing: re-measured $F1$? -0.001 / over-seg 1.16?0.64 (was +0.074 / 1.68?1.01). Now pins the durable direction-of-effect (the gate cuts over-seg) plus the present $\sim F1$ -neutral snapshot. The canonical +0.074 / 6-of-6 / $p=0.031$ win is a property of the retired original-6 corpus (preserved in docs/archives) and is no longer reproduced here.

Re-grounded the module narrative to the present multicourt corpus. The pinned magnitudes are a date-stamped snapshot and will need re-measurement as the golden set grows.

Verified on the local golden files: 5/5 served-litmus tests pass; full suite green (864 passed, cov 81.23%); ruff + mypy clean.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

tests/test_served_gate_a_regression.py | 96 ++++++-----
1 file changed, 58 insertions(+), 38 deletions(-)

22. user (2026-06-19T15:53:06.832Z)

```
133:     n = len(results)
```

23. assistant (2026-06-19T15:53:10.284Z)

Let me look at the actual served_gate_a test to see what it currently asserts about the corpus, since the re-grounding note's "labels are gone / 9 multicourt" is contradicted by the manifest still holding all 6 originals.

24. user (2026-06-19T15:53:10.949Z)

```
1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3     The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4     ``min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5     MagicMock runner). The offline ablation litmus
6     (`test_ablation.test_litmus_reproduces_gate_a`)
7     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8     (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9     runs the real ``FusionSegmenter`` decision seams (`_enrich_candidate_stats` net-
crossings +
10    ``_select_for_handoff`` Gate A) over windows from the **production**
11    ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
12    the served path's measured behaviour.
13
14    GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
15    CI where the golden trajectories are absent; runs on the owner box where they are
present, over
16    **whatever golden clips are currently on disk** (`golden_specs` requires >= 6).
17
18    CORPUS NOTE ? re-grounded 2026-06-18 (the golden set is mutable and grows over time).
The
19    original 6 *single-court* golden clips this litmus was first calibrated on (2026-06-14)
have been
20    retired from the box ? their labels are gone ? so the present golden set is the 9
```

```

**multicourt**
20 clips now in ``output/`` (mahadevpura_1/2, GX0101xx, GX0x0094, Badminton_BXH_2,
Boxhill_Doubles).
21 On that corpus the served path is **recall-starved**: WASB tracks a single shuttle while
several
22 courts rally at once, so ``trajectory_to_action_windows`` emits FEWER windows than there
are
23 rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A only ever
DROPS
24 windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ?
-0.034),
25 not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL windowing
the same
26 gate is ~F1-neutral (?F1 ? -0.001) while still cutting over-segmentation (1.16?0.64).
27
28 The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
29 and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
30 (``docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
31 **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
32 litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
33 re-measurement as the corpus grows.
34 """
35 from __future__ import annotations
36
37 import json
38 import os
39
40 import pytest
41
42 # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
43 # (CI), skip the whole module rather than erroring at import.
44 pytest.importorskip("cv2")
45
46 from backend.eval import served_gate_a as sga # noqa: E402
47 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
48 from backend.pipeline.detectors import rally_gate # noqa: E402
49 from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
50 from backend.eval.metrics import score # noqa: E402
51 from backend.eval import segmentation_metrics as _segm # noqa: E402
52
53 # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
54 # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
55 # out-of-tree checkout.
56 _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
57 MANIFEST = os.environ.get(
58     "SERVED_GATE_A_MANIFEST",
59     os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
60 )
61
62
63 def _golden_specs():
64     """Return the manifest specs whose trajectory + label files are all present, else
[]."""
65     if not os.path.isfile(MANIFEST):
66         return []
67     with open(MANIFEST, encoding="utf-8") as f:
68         specs = json.load(f)

```

```

69         ok = [s for s in specs
70                 if os.path.isfile(str(s.get("trajectory", "")))
71                 and os.path.isfile(str(s.get("labels", "")))]
72         return ok if len(ok) >= 6 else []
73
74
75     _SPECS = _golden_specs()
76     _skip = pytest.mark.skipif(
77         not _SPECS,
78         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
79     )
80
81
82     # ----- served-path
litmus
83
84     @_skip
85     def test_served_path_runs_on_all_golden_videos():
86         """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
EVERY
87         present golden trajectory and yields a handoff window list for each, for both gate
settings.
88         Asserts against the present corpus count (>= 6), not a stale fixed 6 ? the golden
set grows."""
89         results = sga.evaluate_manifest(_SPECS)
90         assert len(results) == len(_SPECS)
91         assert len(results) >= 6
92         for r in results:
93             assert r.num_gt > 0
94             assert r.n_handoff_off >= r.n_handoff_on    # Gate A only ever drops windows,
never adds
95
96
97     @_skip
98     def test_served_gate_a_does_not_materially_regress_f1():
99         """SERVED no-material-regression guard: on the production windowing Gate A must not
MATERIALLY
100         hurt F1. On the present recall-starved multicourt corpus the served path is under-
segmented
101         (over-seg ratio < 1), so Gate A ? which only DROPS windows ? trims a few borderline-
true
102         windows and is mildly negative, NOT the ~neutral it was on the original single-court
6. We pin
103         a band that admits this measured mild loss but trips on a material regression."""
104         results = sga.evaluate_manifest(_SPECS)
105         agg = sga.aggregate(results)
106         # Honest measured value (2026-06-18, present 9-clip multicourt golden set): mean
served
107         # ?F1 ? -0.034 (was ? -0.008 on the retired original-6). Band re-measured to admit
the mild
108         # recall-starved loss while still tripping if Gate A materially destroys served F1.
109         assert -0.06 <= agg["mean_delta_f1"] <= 0.03, agg
110         # And Gate A must never collapse served F1 to ~half the OFF baseline (measured ratio
? 0.83).
111         assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
112
113
114     @_skip
115     def test_served_gate_a_does_not_increase_over_segmentation():
116         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
117         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
118         results = sga.evaluate_manifest(_SPECS)
119         agg = sga.aggregate(results)
120         assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg

```

```

121     # Per-video too: the gate is monotone (kept windows ? all windows).
122     for r in results:
123         assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
124
125
126     @_skip
127     def test_served_uses_production_windowing_operating_point():
128         """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
129         production windowing, this litmus's premise (and the divergence it documents) is
revisited
130         deliberately, not silently."""
131         assert sga.PROD_VELOCITY_THRESH == 0.02
132         assert sga.PROD_MERGE_GAP == 2.0
133         assert sga.PROD_MIN_WINDOW == 2.0
134
135
136     # ----- Gate A under the permissive PROPOSAL
windowing
137
138     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
139     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
140     # small windows). On the original-6 single-court corpus the Gate-A lift here was +0.074
(6/6,
141     # p=0.031; archived). On the present multicourt corpus that lift is GONE ? the gate is
~F1-neutral
142     # but still cuts over-segmentation. We pin the present-corpus behaviour, not the retired
+0.074.
143     _PROPOSAL = (0.005, 1.0, 0.5)
144
145
146     @_skip
147     def test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg():
148         """CONTRAST pin: under the OFFLINE harness's permissive proposal windowing, the SAME
149         ``rally_gate`` net-crossing gate the served seam uses behaves DIFFERENTLY than under
the
150         production windowing. The durable invariant (any corpus): the gate reduces over-
segmentation
151         (seg_on < seg_off), since it only drops windows. On the present multicourt corpus it
is
152         ~F1-neutral (?F1 ? -0.001, was +0.074 on the now-retired original-6) ? proving the
served
153         behaviour is a property of the operating point + corpus, not a broken gate."""
154         d_on, d_off, seg_on, seg_off = [], [], [], []
155         for s in _SPECS:
156             fps, fw = float(s["fps"]), float(s["frame_width"])
157             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
158             wins = TrackNetRunner.trajectory_to_action_windows(
159                 pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
160                 merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
161             iv = [(w["start"], w["end"]) for w in wins]
162             est = rally_gate.estimate_play_axis(pts)
163             gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
164             on = [(g.start, g.end) for g in gated if g.crossings >= 2]
165             gts = load_golden_gts(s["labels"])
166             d_on.append(score(on, gts).f1)
167             d_off.append(score(iv, gts).f1)
168             seg_on.append(_segm.segment_count_ratio(on, gts))
169             seg_off.append(_segm.segment_count_ratio(iv, gts))
170             n = len(d_on)
171             mean_delta = (sum(d_on) - sum(d_off)) / n
172             mean_seg_off, mean_seg_on = sum(seg_off) / n, sum(seg_on) / n
173             # Durable direction-of-effect (corpus-independent): the gate only drops windows, so

```



```

under the
174         # over-segmenting proposal windowing it lowers the over-seg ratio.
175         assert mean_seg_on < mean_seg_off, (mean_seg_on, mean_seg_off)
176         # Honest measured snapshot (2026-06-18, present 9-clip multicourt set): mean ?F1 ?
-0.001,
177         # over-seg 1.16 ? 0.64 (was +0.074 / 1.68 ? 1.01 on the retired original-6).
~F1-neutral band:
178         assert mean_delta == pytest.approx(0.0, abs=0.03), mean_delta
179         assert mean_seg_off == pytest.approx(1.16, abs=0.10), mean_seg_off
180         assert mean_seg_on == pytest.approx(0.64, abs=0.10), mean_seg_on
181

```

25. assistant (2026-06-19T15:53:21.509Z)

Now I understand the full picture. This is critical: the test `\_golden\_specs()` filters the manifest to specs whose **trajectory AND label files** are both present on disk** (lines 69-71). The manifest has 15 entries, but the test only counts those whose files exist on the owner's box.

The re-grounding note (and this test docstring) claims the original-6 single-court clips "have been retired from the box ? their labels are gone" and the present golden set is "9 multicourt clips." But:

1. The manifest STILL lists all 6 originals with label paths pointing to `C:/Users/avidu/Projects/Annotation Setup/...`. Whether those files exist on disk, I cannot verify in this dev container (no GPU/golden files). The manifest is the committed source of truth, and it lists 15 including the 6 originals.
2. The current code's `MULTICOURT\_CLIPS` set (rally_seg_eval.py:51) has only **4** multicourt clips, and the P7 table (which IS in this same doc, computed on the current corpus) reports **11** single-court + 4 multicourt = 15**. So the claim "9 multicourt clips" is contradicted by the doc's own P7 measurement and by the code's MULTICOURT_CLIPS definition.

This is the heart of the problem. Let me carefully assess each proposed fix.

Fix 1 ? "Golden corpus = the 6 labelled matches" ? "the 15 labelled matches".

The manifest has exactly 15 entries. The doc itself later says "n=15 golden" (line 62/68). So "the 6 labelled matches" as a present-tense definition at the top of the doc is genuinely stale. The proposed correction preserves the original 6 enumeration and the owner-confirmation provenance, and adds "since grown to 15." This is accurate and minimal. **CONFIRM Fix 1.**

Let me verify the exact stale_text for Fix 1 matches verbatim including the bullet/newline structure.

26. user (2026-06-19T15:53:22.904Z)

```

18         **Golden corpus = the 6 labelled matches** in `output/human_lovo_manifest.json` (owner-
confirmed
19         2026-06-13: use all 6 ? `adarsh_avi_singles, kushagra_singles, largetest_doubles,
gbaaddy,
20         testlarge_short, mahadevpura_singles`). The model trained on these is what we run
alongside the existing
21         detector on NEW footage for the side-by-side review.

```

27. assistant (2026-06-19T15:53:34.525Z)

Fix 1 stale_text matches verbatim (the proposed stale_text ends at "mahadevpura_singles`").

Now for **Fix 2**. The proposed correction changes "the present golden set is **9** multicourt clips" ? "at the time of this re-grounding the golden set was **9** multicourt clips" (since grown to 15)."

This is the harder call. There are two distinct factual problems with the original re-grounding

note:

1. The "9 multicourt clips" count is stale (now 15 entries).
2. The "multicourt" characterization is wrong (only 4 of the current 15 are multicourt per the code's MULTICOURT_CLIPS; 11 are single-court per P7).

The proposed Fix 2 only addresses problem #1 (the count, recast as historical), preserving the 70.034 measurement provenance. It leaves "multicourt" intact ? and crucially, the note's whole causal argument hinges on "recall-starved corpus (WASB tracks one shuttle while several courts rally)," which is a multicourt-specific claim. The proposed correction's `why\_stale` acknowledges this nuance and deliberately only fixes the size to preserve the measurement's provenance.

However, there's a subtlety I must check: was the corpus EVER "9 multicourt clips"? The re-grounding commit was 2026-06-18. The test docstring lists the 9 as: "mahadevpura_1/2, GX0101xx, GX0x0094, Badminton_BXH_2, Boxhill_Doubles." That's 9 clips at re-grounding time. But the manifest currently has 15 including the 6 single-court originals. The question for Fix 2: is the original-6 actually "retired from the box"?

Let me check whether the original-6 label files might still exist, and re-confirm the manifest contents haven't changed since the re-grounding. Let me check git history of the manifest ? but it's gitignored. Let me verify it's gitignored and check the P7 entry's claim that 11 are single-court.

28. user (2026-06-19T15:53:36.243Z)

```
output/human_lovo_manifest.json
GITIGNORED
```

29. user (2026-06-19T15:53:39.306Z)

```
single (n=11): ['adarsh_avi_singles', 'kushagra_singles', 'largetest_doubles', 'gbaaddy',
'testlarge_short', 'mahadevpura_singles', 'mahadevpura_1', 'GX030094', 'GX010141', 'GX010137',
'GX020094']
multi (n=4): ['mahadevpura_2', 'GX010128', 'Badminton_BXH_2', 'Boxhill_Doubles']
adarsh_avi_singles label exists on disk: False
kushagra_singles label exists on disk: False
largetest_doubles label exists on disk: False
gbaaddy label exists on disk: False
testlarge_short label exists on disk: False
mahadevpura_singles label exists on disk: False
```

30. assistant (2026-06-19T15:53:44.890Z)

This is a very important finding. The original-6 label files do NOT exist on disk (`label exists on disk: False` for all 6). So the re-grounding note's claim "their labels are gone" is actually TRUE for the original-6 on this box ? the label CSVs are absent. But I'm in a dev container that "lacks GPU / torch / cv2 / numpy" and the golden files live outside the repo; the entire `C:/Users/avidu/Projects/Annotation Setup/...` tree may simply not be present in this environment at all. Let me check whether ANY label file exists, to distinguish "labels genuinely retired" from "this container has no golden files at all."

31. user (2026-06-19T15:53:46.664Z)

| | | |
|---------------------|-------------|-----------|
| adarsh_avi_singles | label:False | traj:True |
| kushagra_singles | label:False | traj:True |
| largetest_doubles | label:False | traj:True |
| gbaaddy | label:False | traj:True |
| testlarge_short | label:False | traj:True |
| mahadevpura_singles | label:False | traj:True |
| mahadevpura_2 | label:True | traj:True |
| GX010128 | label:True | traj:True |
| mahadevpura_1 | label:True | traj:True |
| GX030094 | label:True | traj:True |
| Badminton_BXH_2 | label:True | traj:True |

| | | |
|-----------------|------------|-----------|
| GX010141 | label:True | traj:True |
| GX010137 | label:True | traj:True |
| Boxhill_Doubles | label:True | traj:True |
| GX020094 | label:True | traj:True |

32. assistant (2026-06-19T15:54:00.195Z)

This is decisive and surprising ? I'm actually on the owner's box (the golden files ARE present here). The data shows:

- **Original-6** (adarsh_avi_singles ... mahadevpura_singles): trajectory present, but **label files GONE** (`label:False`). The re-grounding note's claim "their labels are gone" is **literally TRUE**.
- **The other 9** (mahadevpura_2, GX010128, mahadevpura_1, GX030094, Badminton_BXH_2, GX010141, GX010137, Boxhill_Doubles, GX020094): both label and trajectory present.

So the litmus's `_golden_specs()` (which requires BOTH trajectory AND labels present) would currently select exactly **9 clips**, NOT 15 ? because the original-6 labels are gone. The re-grounding note's "the present golden set is 9 multicourt clips" refers to **the 9 clips the litmus actually scores**.

But wait ? only 4 of those 9 are in MULTICOURT_CLIPS. Let me check which of the 9 label-present clips are multicourt vs single.

The 9 label-present clips are: mahadevpura_2, GX010128, mahadevpura_1, GX030094, Badminton_BXH_2, GX010141, GX010137, Boxhill_Doubles, GX020094.

Of these, MULTICOURT_CLIPS = {mahadevpura_2, GX010128, Badminton_BXH_2, Boxhill_Doubles} ? only **4 multicourt**. The other 5 (mahadevpura_1, GX030094, GX010141, GX010137, GX020094) are single-court per the code.

So the re-grounding note's characterization **"9 multicourt clips"** is **factually wrong** on the multicourt count even for the 9-clip litmus set: only 4 of those 9 are multicourt. This is genuinely stale/incorrect.

Now I must reconsider the two proposed fixes carefully, because the real situation is nuanced:

Fix 1 (top-of-doc corpus definition): The doc says "Golden corpus = the 6 labelled matches ... use all 6 ? [original 6]". But those original-6 labels are GONE. The model/eval corpus that the doc's own P7/R1 entries use is the 15-entry manifest (n=15 for P7, which scores on trajectory+GT availability differently). Wait ? but P7 reports n=15. Let me check how P7's eval handles the missing original-6 labels.

33. user (2026-06-19T15:54:07.358Z)

Rally-segmentation eval ? windowing 'served' (vt=0.02, merge_gap=2.0, min_win=2.0,

| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio | pred/gt |
|---------------------|-------|---------|--------|--------|------------|-----------|---------|
| adarsh_avi_singles | 0.679 | 0.787 | 0.679 | 0 | 0.00 | 1.30 | 125/96 |
| kushagra_singles | 0.416 | 0.675 | 0.416 | 3 | 0.19 | 1.41 | 45/32 |
| largetest_doubles | 0.667 | 0.800 | 0.667 | 0 | 0.00 | 1.45 | 71/49 |
| gbaaddy | 0.417 | 0.639 | 0.417 | 1 | 0.04 | 2.00 | 96/48 |
| testlarge_short | 0.300 | 0.400 | 0.300 | 0 | 0.00 | 2.33 | 14/6 |
| mahadevpura_singles | 0.602 | 0.747 | 0.602 | 0 | 0.00 | 1.68 | 52/31 |
| mahadevpura_2 | 0.244 | 0.488 | 0.244 | 9 | 0.53 | 1.05 | 42/40 |
| GX010128 | 0.602 | 0.722 | 0.602 | 0 | 0.00 | 1.56 | 81/52 |
| mahadevpura_1 | 0.000 | 0.000 | 0.000 | 2 | 0.80 | 0.20 | 2/10 |
| GX030094 | 0.468 | 0.645 | 0.468 | 0 | 0.00 | 2.02 | 83/41 |
| Badminton_BXH_2 | 0.443 | 0.672 | 0.443 | 2 | 0.09 | 1.77 | 78/44 |
| GX010141 | 0.510 | 0.745 | 0.510 | 6 | 0.45 | 0.76 | 22/29 |
| GX010137 | 0.822 | 0.904 | 0.822 | 0 | 0.00 | 1.15 | 39/34 |
| Boxhill_Doubles | 0.431 | 0.554 | 0.431 | 0 | 0.00 | 2.02 | 87/43 |
| GX020094 | 0.424 | 0.667 | 0.424 | 0 | 0.00 | 1.48 | 59/40 |

```
**Aggregate (n=15, mean [95% CI]):** F1 0.468 [0.367,0.560] · F1@0.25 0.630 [0.516,0.721] ·
F1@0.5 0.468 [0.367,0.560] · merge_rate 0.140 [0.033,0.274] · seg_ratio 1.478 [1.188,1.715]
```

R2 ? tolerance-match (?_start=2.0s, ?_end=1.5s, strict 1:1)

| video | tolF1 | tolP | tolR | ?start | ?end | split | merge |
|---------------------|-------|------|------|--------|-------|-------|-------|
| adarsh_avi_singles | 0.552 | 0.49 | 0.64 | 0.94 | 0.54 | 13 | 0 |
| kushagra_singles | 0.078 | 0.07 | 0.09 | 2.09 | 2.52 | 6 | 3 |
| largetest_doubles | 0.583 | 0.49 | 0.71 | 0.70 | 0.65 | 5 | 0 |
| gbaaddy | 0.278 | 0.21 | 0.42 | 0.93 | 1.10 | 8 | 1 |
| testlarge_short | 0.200 | 0.14 | 0.33 | 2.71 | 2.58 | 1 | 0 |
| mahadevpura_singles | 0.530 | 0.42 | 0.71 | 0.79 | 0.52 | 7 | 0 |
| mahadevpura_2 | 0.122 | 0.12 | 0.12 | 3.06 | 4.60 | 4 | 9 |
| GX010128 | 0.466 | 0.38 | 0.60 | 0.71 | 0.73 | 2 | 0 |
| mahadevpura_1 | 0.000 | 0.00 | 0.00 | 32.58 | 27.00 | 0 | 2 |
| GX030094 | 0.419 | 0.31 | 0.63 | 0.79 | 0.96 | 6 | 0 |
| Badminton_BXH_2 | 0.328 | 0.26 | 0.45 | 1.15 | 0.71 | 4 | 2 |
| GX010141 | 0.431 | 0.50 | 0.38 | 0.78 | 0.68 | 0 | 6 |
| GX010137 | 0.685 | 0.64 | 0.74 | 0.39 | 0.44 | 1 | 0 |
| Boxhill_Doubles | 0.262 | 0.20 | 0.40 | 0.79 | 0.97 | 9 | 0 |

34. assistant (2026-06-19T15:54:13.942Z)

This is a pivotal finding. The stratified eval ran on n=15 **including the original-6** (adarsh_avi_singles, kushagra_singles, etc. all appear with non-zero F1 scores). This means **the original-6 labels DO resolve when `rally\_seg\_eval` runs** ? it found GTs for them. So how do those scores exist if the label files are `label:False`?

The answer: `rally\_seg\_eval` loads golden GTs differently from the served_gate_a litmus. The litmus uses `s["labels"]` (the CSV path, which is gone), but `rally\_seg\_eval` likely uses a separate `golden\_features` JSON for GTs. Let me confirm ? this matters because it determines whether the original-6 are truly "retired."

35. user (2026-06-19T15:54:16.655Z)

```
output\adarsh_avi_singles_golden_features.json
output\Badminton_BXH_2_golden_features.json
output\Boxhill_Doubles_golden_features.json
output\gbaaddy_golden_features.json
output\GX010128_golden_features.json
output\GX010137_golden_features.json
output\GX010141_golden_features.json
output\GX020094_golden_features.json
output\GX030094_golden_features.json
output\kushagra_singles_golden_features.json
output\largetest_doubles_golden_features.json
output\mahadevpura_1_golden_features.json
output\mahadevpura_2_golden_features.json
output\mahadevpura_singles_golden_features.json
output\testlarge_short_golden_features.json
output\adarsh_avi_singles_golden_features.json
output\Badminton_BXH_2_golden_features.json
output\Boxhill_Doubles_golden_features.json
output\fusion_golden.log
output\gbaaddy_golden_features.json
```

36. user (2026-06-19T15:54:17.818Z)

```
17:``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label files.
78:def score_intervals(preds: List[Interval], gts: List[Interval],
100:     gts = filter_by_min_duration(gts, min_duration)
103:     sc = metrics.score(preds, gts, iou_threshold=iou)
104:     f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
```

```

105:     ms = segmentation_metrics.merge_split_report(preds, gts)
106:     tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
107:     be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the R4-aiming
diagnostic)
108:     osr = tm.over_seg_report(preds, gts)
113:         "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
116:         "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
149:     intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
flagged.""
154:         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
155:         gts: List[Interval] = []
158:             gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])]
160:                 "fps": float(v["fps"]), "frame_width": float(v["frame_width"]), "gts":
gts})
199:     returning `(name, court_type, preds, gts)` per video. Lets callers re-score the same
203:         if not v["gts"] or not os.path.exists(v["trajectory"]):
205:             out.append((v["name"], court_type_for(v), windows_for(v, preset, min_crossings),
v["gts"]))
214:     for name, court, preds, gts in predicted:

```

[Showing results with pagination = limit: 20]

37. assistant (2026-06-19T15:54:43.103Z)

Now everything is clear. There are TWO different golden GT sources:

1. ```rally_seg_eval``` uses ``output/<name>_golden_features.json`` (the ``gts`` key) ? and ALL 15 clips have these (including the original-6 like ``adarsh_avi_singles_golden_features.json``). So the original-6 GTs are STILL present for the main eval harness ? that's why P7's n=15 stratified table includes them with real scores.

2. ```served_gate_a` litmus``` uses the raw ``s["labels"]`` CSV path ? and the original-6 CSVs ARE gone, so the litmus only scores the 9 clips that still have CSVs.

So the re-grounding note's claim is **partially true but misleadingly stated**:

- "their labels are gone" ? TRUE only for the **litmus's** CSV-based label loader, but the original-6 GTs are very much alive in the ``golden_features.json`` form used by the main eval (n=15).
- "the present golden set is **9 multicourt clips**" ? this is **doubly wrong**: (a) the broader golden corpus is 15 (P7's own n=15), and even the litmus's 9-clip subset is (b) NOT all multicourt ? only 4 of those 9 are in MULTICOURT_CLIPS.

Now, the critical question for the audit: **which "golden set" does each doc claim refer to, and is the proposed fix correct?**

Fix 1 (line 18): "Golden corpus = the 6 labelled matches" with the original-6 enumerated. This is the doc's top-level definition of the golden corpus used throughout. The corpus is now 15 (P7 n=15, R1 n=13). The proposed correction "the 15 labelled matches" + "the original 6 ... since grown to 15 with the multicourt/Boxhill clips" is accurate (manifest=15, originals retained in the features form, grown with Boxhill/multicourt clips). **CONFIRM Fix 1.**

One nit on Fix 1's corrected_text: it says "since grown to 15 with the multicourt/Boxhill clips." Only 4 of the 9 added are multicourt; most added clips (mahadevpura_1, GX030094, GX010141, GX010137, GX020094) are single-court GoPro clips. "multicourt/Boxhill clips" slightly over-emphasizes multicourt but is defensible as a loose description (the added clips include the Boxhill multicourt set). It's minor and doesn't assert a false count. I'll keep it but this is borderline. Given the instruction to make the smallest accurate edit and the corrected_text is otherwise accurate, I'll mark it **revise** to tighten "multicourt/Boxhill clips" ? "multicourt + GoPro clips" to avoid implying all additions are multicourt. Actually, the original additions span Boxhill multicourt AND single-court GoPro ? let me keep the correction faithful and minimal. I'll revise to "since grown to 15 (Boxhill + multicourt clips added)".

Fix 2 (line 590): "the present golden set is **9 multicourt clips**". The proposed correction recasts the count as historical: "at the time of this re-grounding the golden set was **9 multicourt clips** (since grown to 15)."

But this is problematic. The "9" was never the full golden corpus ? it was specifically the litmus's CSV-label subset (9 clips with CSVs). And those 9 are NOT all multicourt (only 4 are). The proposed correction preserves "9 multicourt clips" as a historical fact, but "9 multicourt clips" was never accurate ? even at re-grounding time, the 9 litmus clips included 5 single-court clips. So recasting it as "at the time ... was 9 multicourt clips" preserves a claim that was wrong-at-the-time (the multicourt characterization), and the "(since grown to 15)" conflates the litmus's 9-clip CSV subset with the 15-clip features corpus ? two different sets.

This is exactly the kind of correction the audit rules warn against: it would introduce a NEW inaccuracy (implying the litmus set grew from 9 to 15, and that those 9 were all multicourt). The ?0.034 measurement is genuinely tied to the litmus's 9-CSV-clip set, which still exists as-is (the original-6 CSVs are still gone ? I verified `label:False`). So in the litmus's own frame, "the present golden set is 9 ... clips" is **still accurate today** (the litmus still scores exactly those 9). The only genuinely-stale part is the **"multicourt" mischaracterization** (only 4 of 9 are multicourt) ? NOT the count and NOT the present-tense.

So Fix 2's proposed correction targets the WRONG thing (it "corrects" the count, which is actually still right for the litmus, and conflates two corpora), while leaving the truly-wrong "multicourt" descriptor in place. This correction would make the doc LESS accurate, not more. Per the rules ("DROP it if ... you cannot verify the evidence" / "the smallest surgical edit"), I should **DROP Fix 2** as proposed.

Let me confirm the litmus still selects exactly 9 (the CSV-present clips) to be certain the count "9" is still current for that note's frame.

38. user (2026-06-19T15:54:47.800Z)

```
litmus _golden_specs count: 9
names: ['mahadevpura_2', 'GX010128', 'mahadevpura_1', 'GX030094', 'Badminton_BXH_2',
'GX010141', 'GX010137', 'Boxhill_Doubles', 'GX020094']
of which multicourt: ['mahadevpura_2', 'GX010128', 'Badminton_BXH_2', 'Boxhill_Doubles']
of which single: ['mahadevpura_1', 'GX030094', 'GX010141', 'GX010137', 'GX020094']
```

39. assistant (2026-06-19T15:55:02.854Z)

Confirmed decisively:

- The **litmus** (`served\_gate\_a` test) `\_golden\_specs()` selects exactly **9 clips** today ? the count "9" in the re-grounding note is **STILL CURRENT** for that note's frame of reference. It has NOT grown to 15 for the litmus.
- Of those 9, only **4 are multicourt** and **5 are single-court** (mahadevpura_1, GX030094, GX010141, GX010137, GX020094). So **"9 multicourt clips"** is wrong on the multicourt descriptor ? it's 9 clips, of which 4 are multicourt.

This means the proposed **Fix 2** is misdirected:

- It "corrects" the count (9 ? recast as historical, "since grown to 15") ? but the count 9 is still correct for the litmus, and the litmus set did NOT grow to 15 (that's the separate features corpus). The proposed correction would introduce a NEW factual error.
- It leaves the actually-stale **"multicourt"** descriptor untouched.

So Fix 2 as proposed must be **DROPPED** ? its corrected_text would make the doc less accurate (conflating the 9-clip litmus set with the 15-clip features corpus, and implying the 9 were all-multicourt-and-grew). The genuine staleness in that sentence is the "multicourt" mischaracterization, which the proposed fix does not address. Since the task says to output only survivors as proposed (confirm/revise the GIVEN fix), and revising Fix 2 into "9 clips (4 multicourt, 5 single)" would be a substantially different fix than proposed, and the proposed direction (count?historical) is itself wrong, I drop it.

Now let me finalize **Fix 1**. Re-checking its corrected_text accuracy: the manifest is 15; the original 6 are retained (their GTs live in `\_golden\_features.json`, used by the n=15 eval); the additions include Boxhill multicourt + GoPro single-court clips. The phrase "since grown to 15 with the multicourt/Boxhill clips" is loose but not false (the additions do include the Boxhill/multicourt clips, among others). To keep it accurate without overclaiming all additions are multicourt, I'll revise that tail slightly. Let me verify the exact corrected_text doesn't misstate. The original 6 enumeration in the correction is verbatim-correct.

I'll mark Fix 1 as ****revise**** with a tightened tail that doesn't imply all 9 additions are multicourt, and confirm the stale_text is an exact unique substring.

Let me do a final exact-match check of Fix 1's full stale_text block.

40. user (2026-06-19T15:55:05.605Z)

count: 1

41. assistant (2026-06-19T15:55:14.823Z)

Fix 1 stale_text is an exact, unique match (count=1).

Final determinations:

- ****Fix 1: CONFIRM (revise)**** ? genuinely stale. "Golden corpus = the 6 labelled matches" with only the original 6 enumerated is contradicted by the 15-entry manifest and the doc's own "n=15 golden" (P7) / "n=13" (R1) entries. The original-6 GTs are retained (in ``*_golden_features.json``, the source ``rally_seg_eval`` uses ? verified by the n=15 stratified run scoring all 6 originals). I'll revise the proposed corrected_text's tail to avoid implying all 9 added clips are multicourt (only 4 are; 5 are single-court GoPro).

- ****Fix 2: DROP**** ? The proposed correction is misdirected and would introduce a NEW inaccuracy. The "9" count refers to the served_gate_a litmus's CSV-label subset, which still selects exactly 9 clips today (verified) ? it has NOT grown to 15 (that's the separate 15-clip ``golden_features`` corpus). The litmus's original-6 CSV labels really are gone (verified ``label:False``), so "their labels are gone" and the present-tense "is 9 ... clips" are accurate for the litmus's frame. The genuinely-stale element is the *****multicourt***** descriptor (only 4 of those 9 are multicourt per ``MULTICOURT_CLIPS`` and the 5 single-court clips in the set), which the proposed fix does not touch. Recasting the count as historical with "(since grown to 15)" conflates two distinct corpora and would be wrong. I drop it rather than substitute a substantially different fix.

42. user (2026-06-19T15:55:23.637Z)

Structured output provided successfully

43. assistant (2026-06-19T15:55:38.560Z)

I verified both proposed fixes against the actual manifest, code, and on-disk golden files. Result: ****1 survivor (Fix 1, revised); Fix 2 dropped.****

****Fix 1 ? CONFIRMED (revised).**** ``docs/QUALITY_ITERATIONS.md:18-20`` defines "Golden corpus = the 6 labelled matches" and enumerates only the original 6. This is genuinely stale: ``output/human_lovo_manifest.json`` now lists ****15**** entries (all 6 originals + 9 additions), and the doc's own later entries already use the grown corpus ? P7 reports ``n=15`` (line 62/68) and R1 ``n=13`` (line 104/109). The original-6 GTs are not lost for the main eval ? they live in ``output/<name>_golden_features.json`` (the source ``backend/eval/rally_seg_eval.py:154-158`` reads), and ``python -m backend.eval.rally_seg_eval --stratify`` scores all 6 originals in its n=15 table. I tightened the proposed correction's tail ("multicourt/Boxhill clips" ? "GoPro/Boxhill clips") because only ****4**** of the 9 added clips are multicourt per ``MULTICOURT_CLIPS`` (``rally_seg_eval.py:51``); the other 5 are single-court, so calling the additions "multicourt" overstates.

****Fix 2 ? DROPPED.**** The re-grounding note at line 590 ("the present golden set is ****9 multicourt clips****") refers to the ****served_gate_a litmus's**** corpus, not the main eval corpus. That litmus (``tests/test_served_gate_a_regression.py``, ``_golden_specs()``) selects manifest entries whose ****CSV label files**** still exist on disk ? and I verified the original-6 CSVs really are gone (``label:False``) while their ``golden_features.json`` GTs remain. So ``_golden_specs()`` still returns ****exactly 9 clips today**** (verified) ? the count "9" and the present tense are ***still correct*** for that note's frame; the litmus set did ****not**** grow to 15 (that is the separate features corpus). The proposed correction ? recasting the count as historical with "(since grown to 15)" ? would conflate the two distinct corpora and introduce a ***new*** inaccuracy. The genuinely-stale element in that sentence is the *****multicourt***** descriptor (only 4 of those 9 are multicourt; 5 are single-court: mahadevpura_1, GX030094,

GX010141, GX010137, GX020094), which the proposed fix does not touch. Since the proposed correction's direction is wrong and a correct fix would be a substantially different edit, I dropped it rather than substitute.

Relevant files:

- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY\_ITERATIONS.md`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally\_seg\_eval.py` (line 51 `MULTICOURT\_CLIPS`; lines 154-158 GT source)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_served\_gate\_a\_regression.py` (`\_golden\_specs()`, the 9-clip litmus set)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\output\human\_lovo\_manifest.json` (15 entries; original-6 label CSVs absent)